OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

Sistemi Operativi: Lezione 6

Indice

1 Memoria Virtuale: Concetti Fondamentali **3**
1.1 L'idea chiave della memoria virtuale 3
1.2 Perché la memoria deve essere in RAM per essere eseguita 4
1.3 Vantaggi dell'esecuzione parziale dei programmi 4
1.4 Perché la memoria virtuale è importante: separazione logico–fisico . . 5
1.5 Struttura dello spazio di indirizzamento virtuale 6
1.6 Memoria virtuale e pagine condivise 7

2 Paginazione su Richiesta (Demand Paging) **9**
2.1 Concetto e motivazione 9
2.2 Il bit di validità e invalidità nella tabella delle pagine 9
2.3 Gestione del page fault 11
2.4 Paginazione su richiesta pura 12
2.5 Riavvio delle istruzioni dopo un page fault 13
2.6 Lista dei frame liberi e azzeramento su richiesta 13
2.7 Impatto sulle prestazioni: tempo di accesso effettivo 14
2.8 Tasso di page fault massimo accettabile 15
2.9 Uso dello swap space nella paginazione su richiesta 15
2.10 Ottimizzazione della creazione di processi: Copy-on-Write 16
2.11 Impatto realistico del tasso di page fault 18

3 Sostituzione delle Pagine (Page Replacement) **19**
3.1 Il problema della sovra-allocazione e la necessità del rimpiazzamento . 19
3.2 Il bit di modifica (dirty bit) 20
3.3 Ruolo della sostituzione nella memoria virtuale 20
3.4 Valutazione degli algoritmi: la stringa di riferimento 21
3.5 Algoritmo FIFO (First-In, First-Out) 22
3.6 L'anomalia di Bélády 23
3.7 Algoritmo Ottimale (OPT / MIN) 23
3.8 Algoritmo LRU (Least Recently Used) 24
3.9 Implementazione di LRU 25
3.10 Approssimazioni di LRU: il bit di riferimento 26
3.11 LRU approssimato: registro di storia degli accessi 26
3.12 Algoritmo della Seconda Chance (Second-Chance) 27
3.13 Algoritmo Clock 27
3.14 Seconda chance migliorata: bit di riferimento e bit di modifica 27

3.15 Algoritmi LFU e MFU 28
3.16 Strategie di supporto alla sostituzione 29

4 Allocazione dei Frame **30**
4.1 Il problema centrale dell'allocazione 30
4.2 Vincoli e requisiti minimi 30
4.3 Allocazione uniforme 31
4.4 Allocazione proporzionale 32
4.5 Sostituzione locale e sostituzione globale 33
4.6 Soglie di frame liberi nella sostituzione globale 33
4.7 Architetture NUMA e allocazione dei frame 34

5 Thrashing e Modelli di Controllo **36**
5.1 Definizione e cause del thrashing 36
5.2 Thrashing e utilizzazione della CPU 37
5.3 Soluzione locale vs globale al thrashing 37
5.4 Il modello della località 38
5.5 Il modello del working set 38
5.6 Implementazione approssimata del working set 39
5.7 Controllo della frequenza di page fault (PFF) 40
5.8 Compressione della memoria 41

6 Allocazione della Memoria del Kernel **43**
6.1 Perché il kernel gestisce la memoria diversamente 43
6.2 Il Buddy System 44
6.3 Lo Slab Allocator 46
6.4 Vantaggi dello Slab Allocator 47

1 Memoria Virtuale: Concetti Fondamentali

1.1 L'idea chiave della memoria virtuale

Per comprendere la memoria virtuale è necessario partire da una domanda elementare: *cosa succede quando un programma è più grande della memoria fisica disponibile?* Per decenni questa è stata una limitazione reale: il programmatore doveva spezzare manualmente il codice in segmenti (*overlay*) e gestirne il caricamento. La memoria virtuale risolve questo problema in modo trasparente, spostando la complessità dal programmatore al sistema operativo e all'hardware.

Definizione

La **memoria virtuale** è una tecnica che consente ai processi di essere eseguiti anche se non sono caricati interamente nella memoria fisica (RAM). Ogni processo dispone di uno **spazio di indirizzamento virtuale** che può essere molto più grande della RAM effettivamente installata. L'hardware e il sistema operativo si occupano di mappare, dinamicamente e in modo trasparente, le porzioni dello spazio virtuale sulle porzioni fisiche effettivamente necessarie in un dato istante.

L'idea chiave su cui si fonda l'intera costruzione è la seguente: *in ogni momento un processo usa attivamente soltanto una piccola frazione del proprio codice e dei propri dati.* Le routine di gestione degli errori, ad esempio, vengono raramente eseguite; le strutture dati di grandi dimensioni sono spesso sparse e utilizzate solo parzialmente; grandi sezioni di codice possono non essere mai raggiunte durante una particolare esecuzione. Pertanto, caricare in RAM soltanto le pagine attualmente necessarie è non solo possibile, ma efficiente.

I benefici principali che derivano da questa tecnica sono i seguenti.

- **Superamento del limite fisico della RAM.** I programmi non sono più vincolati alla quantità di RAM installata. Un processo può indirizzare, in linea di principio, uno spazio logico dell'ordine dei terabyte su una macchina con soli pochi gigabyte di RAM.
- **Isolamento del programmatore dai dettagli hardware.** Lo sviluppatore scrive il codice come se disponesse di una grande memoria contigua, senza preoccuparsi dei limiti fisici.
- **Maggiore numero di processi in esecuzione simultanea.** Poiché ogni processo occupa meno RAM di quanto richiederebbe se fosse caricato interamente, il sistema operativo può tenere in esecuzione più processi contemporaneamente, migliorando l'utilizzo della CPU e il *throughput* complessivo.
- **Supporto alla memoria condivisa e alla condivisione di file.** Più processi possono condividere le stesse pagine fisiche (ad esempio le librerie condivise), risparmiando memoria.
- **Creazione efficiente di processi (copy-on-write).** Alla creazione di un processo figlio non è necessario duplicare immediatamente l'intero spazio di indirizzamento del padre; le pagine vengono copiate fisicamente solo quando vengono modificate.

- **Riduzione delle operazioni di I/O.** Caricare solo le pagine necessarie riduce le letture da disco, accorciando i tempi di avvio e di esecuzione dei programmi.

Attenzione

La memoria virtuale non è priva di costi. Richiede un supporto hardware dedicato (la *Memory Management Unit*, MMU) e una gestione accurata da parte del sistema operativo. Una gestione inefficiente — in particolare un'eccessiva frequenza di *page fault* — può causare un peggioramento drastico delle prestazioni, fenomeno noto come **thrashing**, che verrà trattato in dettaglio nelle sezioni successive.

1.2 Perché la memoria deve essere in RAM per essere eseguita

Prima di analizzare come funziona la memoria virtuale è necessario chiarire il vincolo di base da cui parte la sua motivazione.

La CPU può eseguire istruzioni *solo* se esse si trovano nella memoria fisica. Questo non è un dettaglio implementativo trascurabile: il bus degli indirizzi fisici collega la CPU direttamente alla RAM; ogni ciclo di *fetch* di un'istruzione opera su indirizzi fisici reali. Se il codice non è in RAM, il processore non può accedervi direttamente.

L'approccio più semplice, usato nei sistemi più primitivi, consiste nel caricare l'intero processo in memoria prima di avviarlo. Questa strategia presenta un limite evidente: la dimensione di un programma è vincolata alla quantità di RAM disponibile. Un programma da 2 GB non può girare su una macchina con 1 GB di RAM.

Il **caricamento dinamico** (*dynamic loading*) è un primo passo verso il superamento di questo limite: il codice viene caricato in memoria solo quando viene chiamato per la prima volta. Tuttavia questa tecnica aggiunge complessità al programmatore (che deve organizzare il codice in moduli caricabili separatamente) e non risolve il problema fondamentale in modo trasparente.

La memoria virtuale offre la soluzione trasparente: solo le porzioni del programma effettivamente usate in un dato momento vengono mantenute in RAM, mentre il resto risiede su disco.

Osservazione

L'osservazione empirica che giustifica la memoria virtuale è che, nella pratica, solo una piccola parte di un programma è attiva in ogni istante. Codice per la gestione degli errori, dati raramente usati, interi moduli mai invocati in un'esecuzione specifica — tutto questo non ha bisogno di occupare RAM preziosa.

1.3 Vantaggi dell'esecuzione parziale dei programmi

Da quanto detto, i vantaggi dell'esecuzione parziale si possono riassumere con maggiore precisione.

Dimensione dei programmi svincolata dalla RAM. Un programma può avere uno spazio di indirizzamento molto più grande della memoria fisica. Il sistema operativo carica in RAM solo le pagine necessarie; le altre restano su disco (nello *swap space* o nel file eseguibile stesso) e vengono caricate su richiesta.

Maggiore multiprogrammazione. Se ogni processo occupa in media solo una frazione della propria dimensione virtuale, la RAM fisica può ospitare più processi contemporaneamente. Ciò aumenta l'utilizzo della CPU (meno attese per I/O) e il *throughput* del sistema.

Riduzione delle operazioni di I/O al caricamento. All'avvio di un programma non è necessario trasferire l'intero eseguibile dal disco alla RAM. Si carica solo ciò che serve subito, il che riduce il tempo di avvio.

Richiami sui prerequisiti: paginazione. La memoria virtuale si appoggia tipicamente al meccanismo di **paginazione**: sia lo spazio logico che quello fisico vengono suddivisi in blocchi di dimensione fissa chiamati, rispettivamente, **pagine** (lato logico) e **frame** (lato fisico). La **tabella delle pagine** (*page table*) associa a ogni numero di pagina logica il numero di frame fisico corrispondente.

1.4 Perché la memoria virtuale è importante: separazione logico–fisico

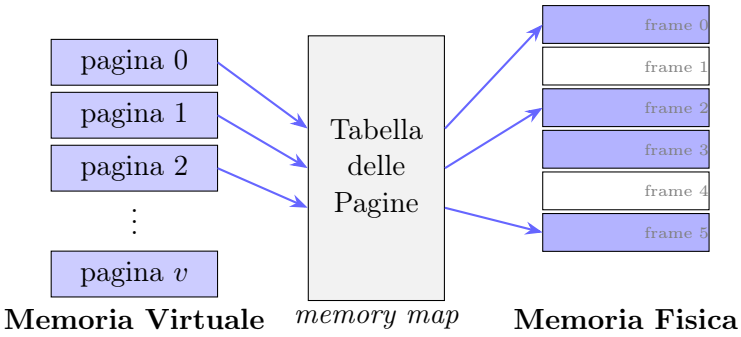
Il contributo concettuale più profondo della memoria virtuale è la **separazione tra spazio di indirizzamento logico e spazio di indirizzamento fisico**. Questa separazione ha conseguenze profonde sia per il programmatore che per il sistema operativo.

Definizione

Lo **spazio di indirizzamento logico** (o virtuale) è l'insieme degli indirizzi che un processo può generare. Ogni processo ha il proprio spazio logico, che inizia tipicamente dall'indirizzo 0 e si estende fino a un massimo determinato dall'architettura (es. 2^{64} byte su sistemi a 64 bit). Questo spazio è *privato* al processo: indirizzi logici uguali in processi diversi corrispondono a frame fisici distinti.
Lo **spazio di indirizzamento fisico** è l'insieme degli indirizzi reali della RAM installata nel sistema. La sua dimensione è fissa e coincide con la quantità di RAM presente.

La **Memory Management Unit (MMU)** è il componente hardware che traduce, ad ogni accesso alla memoria, un indirizzo logico nel corrispondente indirizzo fisico. Questa traduzione avviene consultando la tabella delle pagine del processo corrente. Se la pagina richiesta non si trova in RAM, la MMU genera un'eccezione hardware (*page fault*) che trasferisce il controllo al sistema operativo.

Il diagramma concettuale è il seguente:



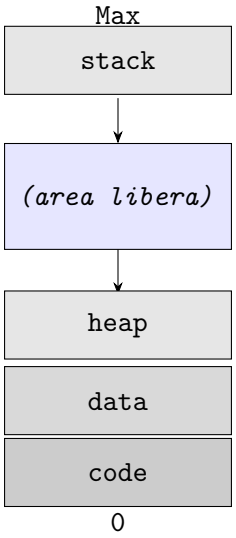
Come si osserva nella figura, lo spazio logico del processo appare come un insieme contiguo di pagine numerate da 0 a v. Nella memoria fisica, tuttavia, le pagine sono sparse in frame non necessariamente contigui. La *memory map* (tabella delle pagine) registra quale frame fisico contiene ciascuna pagina logica. Le pagine logiche che non si trovano in RAM (caselle bianche nella colonna destra) sono memorizzate su disco e verranno caricate solo quando necessario.

Osservazione

Un vantaggio fondamentale di questa astrazione è che **la memoria fisica non deve essere contigua**. Il sistema operativo può usare qualsiasi frame libero, ovunque si trovi nella RAM, per ospitare una pagina logica. Questo elimina del tutto il problema della frammentazione esterna che affligge i sistemi con allocazione contigua.

1.5 Struttura dello spazio di indirizzamento virtuale

Lo spazio di indirizzamento virtuale di un processo moderno ha una struttura ben precisa, progettata per supportare sia la crescita dinamica dei dati che il caricamento di librerie condivise a tempo di esecuzione.



La struttura, dall'indirizzo più basso al più alto, è la seguente.

- **Segmento codice (`code`).** Contiene le istruzioni eseguibili del programma. È tipicamente di sola lettura e ha dimensione fissa determinata al momento della compilazione.
- **Segmento dati (`data`).** Contiene le variabili globali e statiche inizializzate. Anche questo ha dimensione fissa.
- **Heap.** Contiene la memoria allocata dinamicamente (tramite `malloc`, `new` e simili). Cresce verso l'*alto* (verso indirizzi più grandi) man mano che il programma alloca nuova memoria.
- **Area libera (sparsità).** Fra heap e stack esiste un'ampia regione di indirizzi virtuali non mappati. Questa regione è la chiave dell'efficienza: gli indirizzi esistono nello spazio logico ma *non corrispondono a nessun frame fisico* finché non vengono effettivamente usati. Si parla di **spazio di indirizzamento sparso** (*sparse address space*).
- **Stack.** Contiene le variabili locali ai sottoprogrammi e le informazioni di chiamata (*stack frame*). Cresce verso il *basso* (verso indirizzi più piccoli) a ogni chiamata di funzione e si riduce a ogni ritorno.

Osservazione

La coesistenza di heap che cresce verso l'alto e stack che cresce verso il basso con un'ampia zona libera in mezzo è una scelta progettuale elegante: entrambe le strutture possono espandersi liberamente senza necessità di rilocazione, fino a che non si incontrano (evento che causerebbe un errore di *stack overflow* o di esaurimento della memoria heap).
Nell'area libera possono inoltre essere mappate le **librerie condivise** a tempo di esecuzione (*shared libraries*, file `.so` su Linux o `.dll` su Windows): esse vengono inserite nello spazio virtuale del processo dal linker dinamico senza occupare spazio nel file eseguibile.

1.6 Memoria virtuale e pagine condivise

Uno dei vantaggi più potenti della memoria virtuale è la possibilità di fare in modo che **la stessa pagina fisica appaia in più spazi di indirizzamento virtuali** distinti, con conseguente risparmio di memoria e facilitazione della comunicazione tra processi.

Definizione

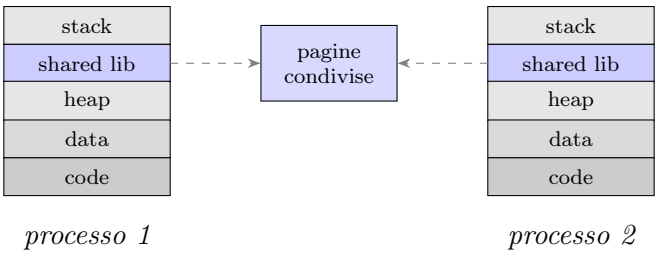
Una **pagina condivisa** (*shared page*) è un frame fisico che è mappato, tramite le rispettive tabelle delle pagine, nello spazio virtuale di due o più processi distinti. Ogni processo vede la pagina al proprio indirizzo virtuale (che può essere diverso da processo a processo), ma l'unica copia fisica è quella nel frame condiviso.

Questo meccanismo si applica in tre contesti principali.

Librerie condivise (*shared libraries*). Librerie di uso comune come la libreria C standard (`libc`) vengono caricate in RAM una sola volta. Tutti i processi che le utilizzano le mappano nel proprio spazio virtuale come pagine di sola lettura. In questo modo, se 100 processi usano la stessa libreria da 2 MB, in RAM occupano comunque soli 2 MB (invece di 200 MB).

Regioni di memoria condivisa (*shared memory*). Usate per la **comunicazione inter-processo** (IPC) ad alta velocità. Due o più processi possono scambiarsi dati semplicemente leggendo e scrivendo in una regione di memoria comune, evitando costose chiamate di sistema. Ogni processo vede la regione come parte del proprio spazio virtuale, all'indirizzo che più gli è conveniente.

Creazione di processi con `fork()`. Quando un processo crea un figlio con la chiamata di sistema `fork()`, i due processi hanno inizialmente spazi di indirizzamento *identici*. Invece di copiare fisicamente tutte le pagine del padre — operazione costosa e spesso inutile — il sistema operativo fa sì che entrambi i processi puntino agli stessi frame fisici. Le pagine sono marcate come **copy-on-write** (CoW): solo quando uno dei due processi *modifica* una pagina, il sistema operativo crea una copia privata di quel frame per il processo modificante. Le pagine non modificate rimangono condivise e non vengono mai duplicate. Questo meccanismo viene approfondito nella sezione dedicata alla copy-on-write.



Il diagramma mostra due processi con strutture virtuali analoghe. La regione "shared lib" di entrambi punta alle *stesse pagine fisiche* nella zona centrale; le altre regioni (stack, heap, data, code) sono private e fisicamente distinte.

Nota

La condivisione di pagine richiede attenzione alla coerenza: le pagine condivise in sola lettura (come il codice di una libreria) sono sempre sicure; le pagine condivise in lettura-scrittura (come le regioni IPC) richiedono meccanismi di sincronizzazione a carico del programmatore (mutex, semafori, ecc.) per evitare condizioni di gara (*race conditions*).

2 Paginazione su Richiesta (Demand Paging)

2.1 Concetto e motivazione

Nei sistemi con memoria virtuale, la domanda naturale è: *quando deve essere caricata in RAM una pagina?* La risposta più immediata sarebbe caricare tutte le pagine di un processo al momento del suo avvio. Tuttavia, abbiamo già visto che la maggior parte delle pagine non viene mai usata durante una particolare esecuzione. La **paginazione su richiesta** (*demand paging*) adotta una strategia opposta e più efficiente: una pagina viene caricata in RAM *solo quando il processo tenta di accedervi per la prima volta.*

Definizione

La **paginazione su richiesta** è il meccanismo con cui le pagine di un processo vengono caricate nella memoria fisica *on demand*, cioè soltanto al momento del primo accesso. Le pagine non ancora accedute restano su disco e non occupano alcun frame fisico. Questo meccanismo costituisce il cuore operativo della memoria virtuale nei sistemi operativi moderni.

I vantaggi rispetto al caricamento integrale sono diretti:

- il codice e i dati mai acceduti (routine di errore, dati inizializzati ma non usati, ecc.) non consumano mai RAM;
- il processo può avviarsi quasi immediatamente, senza attendere il caricamento dell'intero eseguibile;
- la RAM risparmata può essere assegnata ad altri processi, aumentando il grado di multiprogrammazione.

Osservazione

La paginazione su richiesta non è un'ottimizzazione facoltativa: è il meccanismo che *rende possibile* l'esistenza stessa della memoria virtuale come la conosciamo. Senza di essa, ogni processo dovrebbe essere interamente residente in RAM, vanificando tutti i benefici discussi nella sezione precedente.

2.2 Il bit di validità e invalidità nella tabella delle pagine

Per realizzare la paginazione su richiesta è necessario che la tabella delle pagine possa distinguere le pagine già caricate in RAM da quelle ancora su disco. A tale scopo ogni voce della tabella delle pagine (*Page Table Entry*, PTE) contiene un **bit di validità/invalidità** (*valid/invalid bit*).

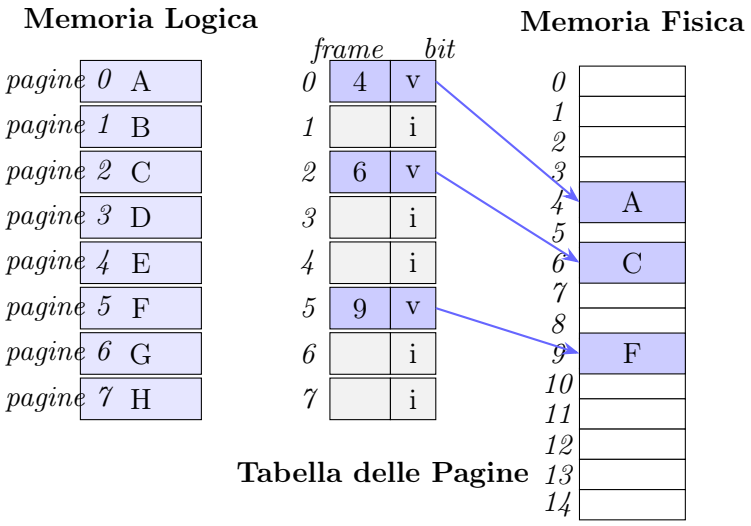
Definizione

Il **bit di validità** di una voce della tabella delle pagine assume due valori:

- **Valido (`v`):** la pagina è attualmente presente in un frame della memoria fisica. L'accesso può procedere normalmente; il campo "numero di frame" della PTE contiene l'indirizzo fisico corretto.
- **Non valido (`i`):** la pagina *non* è in RAM. Può trovarsi su disco (swap space o file eseguibile) oppure l'indirizzo potrebbe essere illegale. Un accesso a una pagina con bit non valido genera un **page fault**.

La CPU, a ogni accesso alla memoria, consulta la PTE corrispondente alla pagina referenziata. Se il bit è valido, la traduzione dell'indirizzo prosegue normalmente verso la RAM. Se il bit è non valido, la MMU genera un'eccezione di tipo *page fault*, trasferendo il controllo al gestore del sistema operativo.

Il seguente schema illustra il funzionamento con un esempio concreto. Si considerino 8 pagine logiche (A–H) e una tabella delle pagine con 6 voci:



Le pagine A, C ed F hanno bit valido e sono presenti nei frame 4, 6 e 9 rispettivamente. Le pagine B, D, E, G, H hanno bit non valido: non sono in RAM. Un accesso a una qualsiasi di queste ultime provocherà un page fault.

Nota

Il bit di validità è distinto dal bit di protezione (lettura, scrittura, esecuzione). Una pagina può essere valida ma protetta in scrittura (ad esempio le pagine di codice di una libreria condivisa). Un accesso in scrittura a una pagina protetta genera un'eccezione diversa dal page fault.

2.3 Gestione del page fault

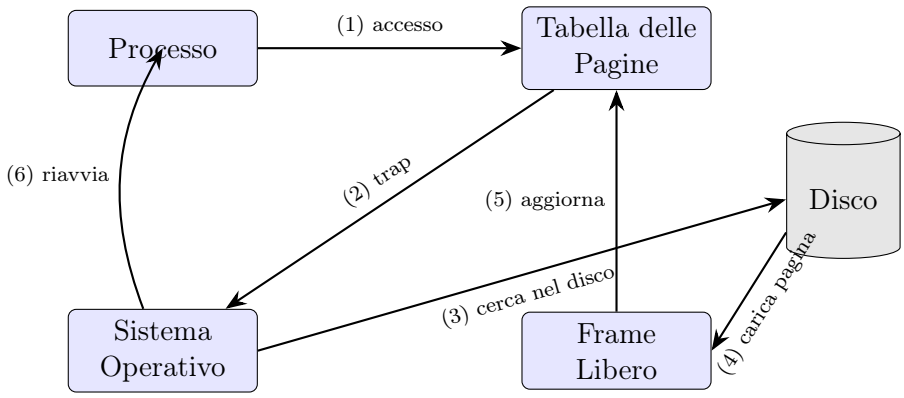
Il **page fault** è l'evento centrale attorno al quale ruota l'intera meccanica della paginazione su richiesta. Vediamo in dettaglio cosa accade, passo per passo, quando si verifica.

Definizione

Un **page fault** è un'eccezione hardware generata dalla MMU quando un processo tenta di accedere a una pagina il cui bit di validità è non valido, ovvero una pagina non attualmente presente in RAM. Il page fault trasferisce il controllo al sistema operativo, che si occupa di risolvere la situazione caricando la pagina mancante dal disco, oppure di terminare il processo se l'accesso è illegale.

La sequenza di operazioni è la seguente, con riferimento ai passi numerati nel diagramma:

1. **Riferimento alla memoria.** Il processo tenta di accedere a un indirizzo logico (ad es. `load M`). La MMU consulta la tabella delle pagine e trova il bit non valido (`i`).
2. **Trap al sistema operativo.** La MMU genera un'eccezione di page fault. Il controllo passa al gestore del page fault del sistema operativo, salvando automaticamente lo stato del processo (registri, program counter, flag).
3. **Verifica della legittimità.** Il sistema operativo controlla se il riferimento è legale:
   - Se l'accesso è *illegale* (ad es. indirizzo fuori dallo spazio virtuale del processo, accesso in scrittura a una pagina di sola lettura) → il processo viene terminato con un segnale di errore (es. segmentation fault).
   - Se l'accesso è *valido* (la pagina esiste ma non è in RAM) → si procede al caricamento.
4. **Caricamento della pagina dal disco.** Il sistema operativo individua la pagina nello *swap space* o nel file eseguibile sul disco e avvia un'operazione di I/O per leggerla.
5. **Assegnazione di un frame libero.** Mentre (o dopo che) la pagina viene letta dal disco, il sistema operativo individua un frame fisico libero dove collocarla. Se non esistono frame liberi, viene eseguito un algoritmo di *page replacement* (trattato nelle sezioni successive) per liberarne uno.
6. **Aggiornamento della tabella delle pagine.** Una volta caricata la pagina nel frame, il sistema operativo aggiorna la PTE: inserisce il numero del frame assegnato e imposta il bit di validità a `v`.
7. **Riavvio dell'istruzione interrotta.** Il processo viene riportato nello stato in cui si trovava al momento del page fault e l'istruzione che ha causato l'eccezione viene *riavviata* (non ripresa da metà, ma rieseguita dall'inizio). Questa volta la pagina è in RAM e l'accesso avrà successo.



Osservazione

Il riavvio dell'istruzione interrotta (passo 7) è un requisito fondamentale per la correttezza del meccanismo. Il sistema operativo deve salvare *completamente* lo stato del processo al momento del fault (tutti i registri, il program counter, i flag di condizione) in modo da poter ripristinare esattamente quella situazione prima di rieseguire l'istruzione. Qualsiasi modifica allo stato del processo avvenuta *parzialmente* durante l'istruzione che ha causato il fault deve essere annullata (*rolled back*).

2.4 Paginazione su richiesta pura

Una variante estrema della paginazione su richiesta è la **paginazione su richiesta pura** (*pure demand paging*): il processo viene avviato con *zero pagine in RAM*. Nessuna pagina viene precaricata.

- Al primo ciclo di *fetch*, la CPU tenta di leggere la prima istruzione: la pagina non è in RAM → page fault.
- Il sistema operativo carica quella pagina; l'esecuzione riprende.
- Man mano che il processo esegue, ogni nuova pagina referenziata per la prima volta genera un page fault e viene caricata.
- Solo le pagine effettivamente accedute entrano mai in RAM.

Questo approccio massimizza il risparmio di memoria, ma può causare molti page fault nelle prime fasi di esecuzione. Il numero di fault tende a diminuire rapidamente una volta che le pagine più usate sono state caricate, grazie alla *località dei riferimenti*.

Requisiti hardware. La paginazione su richiesta pura richiede:

- Una tabella delle pagine con bit di validità per segnalare le pagine non in RAM.
- Una memoria secondaria veloce (*swap space*) su cui conservare le pagine assenti dalla RAM, organizzata in modo da minimizzare i tempi di accesso.

2.5 Riavvio delle istruzioni dopo un page fault

Il riavvio corretto di un'istruzione interrotta da un page fault presenta alcune complessità che è utile esaminare nel dettaglio.

Il sistema operativo deve salvare lo stato completo del processo (tutti i registri, il program counter, i flag). Dopo aver caricato la pagina mancante, lo stato viene ripristinato e l'istruzione viene rieseguita dall'inizio.

Si presentano due casi principali.

Page fault durante il fetch dell'istruzione. Se il fault avviene mentre la CPU sta leggendo l'istruzione stessa (la pagina contenente il codice non è in RAM), la soluzione è semplice: si carica la pagina e si riesegue il fetch. Non vi è alcuno stato parzialmente modificato da annullare.

Page fault durante il fetch degli operandi. Se l'istruzione è già stata decodificata e il fault avviene mentre si accede a un operando in memoria, la situazione è più delicata. Alcune architetture CISC consentono istruzioni che modificano più locazioni di memoria in un singolo ciclo (ad es. istruzioni di copia a blocchi). Se il fault avviene a metà di tale operazione, alcune locazioni sono già state modificate. L'istruzione deve essere riavviata dall'inizio, ma le modifiche parziali devono essere annullate prima, altrimenti la memoria si troverebbe in uno stato incoerente. Le architetture moderne risolvono questo problema con meccanismi hardware dedicati (registri temporanei, operazioni idempotenti, ecc.).

Nota

Questo è uno dei motivi per cui la progettazione dell'ISA (*Instruction Set Architecture*) deve tenere conto fin dall'inizio del supporto alla memoria virtuale. Un'architettura che non consente il riavvio atomico di tutte le sue istruzioni rende la paginazione su richiesta molto più complessa da implementare.

2.6 Lista dei frame liberi e azzeramento su richiesta

Per soddisfare rapidamente i page fault, il sistema operativo mantiene una struttura dati apposita.

Definizione

La **lista dei frame liberi** (*free-frame list*) è una lista concatenata che contiene i numeri di tutti i frame fisici attualmente non assegnati ad alcuna pagina. Quando si verifica un page fault e la pagina deve essere caricata, il sistema operativo preleva il primo frame dalla lista e lo usa come destinazione della lettura da disco.

All'avvio del sistema, tutti i frame non occupati dal kernel vengono inseriti nella lista. Man mano che i processi generano page fault, la lista si svuota. Quando diventa troppo corta, il sistema operativo deve liberare dei frame tramite il meccanismo di *page replacement* (sezione successiva).

Azzeramento su richiesta (*zero-fill-on-demand*). Prima di assegnare un frame a una nuova pagina, il sistema operativo lo **azzera**, cioè scrive zero in ogni byte del frame. Questo passaggio è fondamentale per la sicurezza: senza l'azzeramento, il nuovo processo potrebbe leggere dati residui lasciati dal processo precedente che occupava quel frame, violando l'isolamento tra processi. L'azzeramento comporta un piccolo costo in termini di tempo, ma è irrinunciabile.

Osservazione

Alcuni sistemi operativi azzèrano i frame in anticipo, durante i periodi di inattività della CPU, in modo da avere sempre una scorta di frame già azzerati pronti per l'uso. Questo riduce la latenza percepita dal processo al momento del page fault.

2.7 Impatto sulle prestazioni: tempo di accesso effettivo

Il costo di un page fault è dominato dal tempo necessario per leggere una pagina dal disco, un'operazione che richiede diversi ordini di grandezza più tempo di un normale accesso alla RAM. È essenziale quantificare questo impatto.

Definizione

Il **tempo di accesso effettivo alla memoria** (*Effective Access Time*, EAT) in un sistema con paginazione su richiesta è definito come:

$$\mathrm{EAT} = (1 - p) \times t_{\mathrm{ma}} + p \times t_{\mathrm{pf}} \tag{1}$$

dove:

- $p \in [0, 1]$ è il **tasso di page fault** (*page-fault rate*): la probabilità che un accesso alla memoria provochi un page fault;
- t_{ma} è il tempo di accesso alla memoria senza page fault (tipicamente $\approx 200\,\mathrm{ns}$);
- t_{pf} è il tempo di servizio di un page fault, che comprende il salvataggio dello stato del processo, la lettura della pagina dal disco, l'aggiornamento della tabella delle pagine e il riavvio dell'istruzione (tipicamente $\approx 8\,\mathrm{ms} = 8\,000\,000\,\mathrm{ns}$).

Interpretazione della formula. La formula (1) è una media pesata: con probabilità $(1-p)$ l'accesso avviene senza fault e costa t_{ma}; con probabilità p si verifica un page fault e il costo totale è t_{pf} (che include anche la lettura normale della pagina al suo interno). Il termine $(1 - p) \times t_{\mathrm{ma}}$ è trascurabile rispetto a $p \times t_{\mathrm{pf}}$ anche per valori molto piccoli di p, data la differenza di quattro ordini di grandezza tra t_{ma} e t_{pf}.

Esempio

Con $t_{\text{ma}} = 200\,\text{ns}$ e $t_{\text{pf}} = 8\,\text{ms} = 8\,000\,000\,\text{ns}$, per $p = 0{,}001$ (un page fault ogni 1000 accessi):

$$\text{EAT} = 0{,}999 \times 200 + 0{,}001 \times 8\,000\,000 \approx 199{,}8 + 8\,000 \approx 8\,200\,\text{ns} = 8{,}2\,\mu\text{s}$$

Il tempo di accesso effettivo è circa **41 volte maggiore** del tempo di accesso normale. Un tasso di page fault di solo lo 0,1% degrada le prestazioni di un fattore 41.

Attenzione

Questo risultato mostra con chiarezza che i sistemi a paginazione su richiesta sono *estremamente sensibili* al tasso di page fault. Anche tassi apparentemente trascurabili hanno un impatto devastante sulle prestazioni. Il contenimento del tasso di page fault non è un'ottimizzazione secondaria: è un requisito fondamentale per la corretta operatività del sistema.

2.8 Tasso di page fault massimo accettabile

Dati i numeri della sezione precedente, è possibile calcolare il tasso di page fault massimo affinché il degrado delle prestazioni rimanga entro una soglia accettabile.

Obiettivo: degrado inferiore al 10%. Si vuole che l'EAT non superi il 10% in più del tempo di accesso normale:

$$\text{EAT} < 1{,}1 \times t_{\text{ma}} = 1{,}1 \times 200 = 220\,\text{ns}$$

Sostituendo nella formula (1):

$$220 > 200 + 7\,999\,800 \times p$$

$$p < \frac{20}{7\,999\,800} \approx 2{,}5 \times 10^{-6}$$

Osservazione

Un tasso $p \approx 2{,}5 \times 10^{-6}$ significa che si può tollerare al massimo **1 page fault ogni 400 000 accessi alla memoria**. Per un sistema che esegue miliardi di accessi al secondo, questo corrisponde a non più di qualche migliaia di page fault al secondo. Mantenere questo standard richiede algoritmi di rimpiazzamento delle pagine accurati e un'allocazione adeguata dei frame a ciascun processo — argomenti trattati nelle sezioni seguenti.

2.9 Uso dello swap space nella paginazione su richiesta

Lo **swap space** è un'area della memoria secondaria (tipicamente una partizione dedicata del disco o un file di grandi dimensioni) riservata dal sistema operativo per contenere le pagine che non si trovano in RAM. La sua gestione ha un impatto diretto sulle prestazioni del sistema.

Perché lo swap space è più veloce del file system. Le operazioni di I/O verso lo swap space sono generalmente più veloci di quelle verso il file system ordinario per due ragioni:

1. Lo swap space è organizzato in blocchi contigui, eliminando la frammentazione tipica dei file system.
2. Non richiedono operazioni di directory lookup, gestione dei metadati, journaling o altri overhead del file system.

Esistono due strategie principali per l'uso dello swap space.

Strategia 1: copia integrale all'avvio. All'avvio del processo, l'intero eseguibile viene copiato nello swap space. Tutti i page fault futuri leggono sempre dallo swap, che è più veloce del file system. Svantaggio: il tempo di avvio è più lungo e lo swap space deve essere abbastanza grande da contenere tutti i programmi attivi.

Strategia 2: caricamento lazy con promozione allo swap. Le pagine vengono caricate direttamente dal file eseguibile (o dalle librerie) al primo accesso. Solo quando vengono *espulse* dalla RAM per la prima volta vengono scritte nello swap space. I fault successivi le leggono sempre dallo swap. Questo riduce il tempo di avvio e usa lo swap solo per le pagine effettivamente necessarie.

Nota

I sistemi operativi moderni (Linux, macOS, Windows) usano varianti della seconda strategia. Le pagine di codice di sola lettura non vengono mai scritte nello swap perché possono sempre essere ricaricate dal file eseguibile originale (che per definizione non cambia). Solo le pagine di dati modificate (*dirty pages*) devono essere salvate nello swap prima di essere espulse dalla RAM.

2.10 Ottimizzazione della creazione di processi: Copy-on-Write

La creazione di un nuovo processo tramite `fork()` è un'operazione frequente nei sistemi Unix-like. Senza ottimizzazioni, `fork()` dovrebbe duplicare l'intero spazio di indirizzamento del processo padre — un'operazione costosa sia in termini di tempo che di memoria, spesso inutile perché il figlio eseguirà immediatamente `exec()` sostituendo il proprio spazio di indirizzamento.

Definizione

La tecnica **copy-on-write** (CoW) consente al processo padre e al processo figlio di condividere inizialmente le stesse pagine fisiche dopo una `fork()`. Le pagine condivise sono marcate come *copy-on-write* nella tabella delle pagine di entrambi i processi. Solo quando uno dei due processi tenta di **scrivere** su una pagina condivisa, il sistema operativo crea una copia privata di quella pagina per il processo scrivente. Le pagine mai modificate restano condivise per tutta la vita dei due processi.

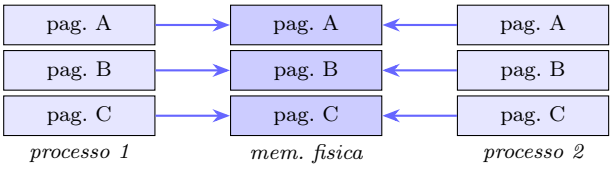
Il meccanismo funziona come segue:

1. `fork()` crea il processo figlio copiando *solo* la tabella delle pagine del padre (non i frame), e marca tutte le pagine come copy-on-write. Entrambi i processi puntano agli stessi frame fisici.

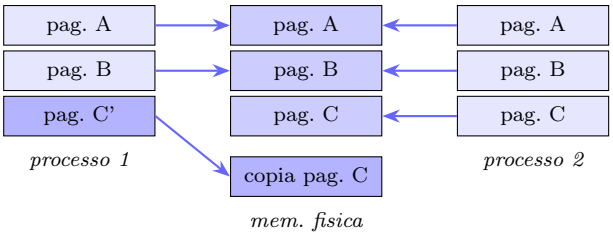
2. Se un processo *legge* una pagina condivisa: nessuna azione necessaria, l'accesso avviene normalmente.

3. Se un processo *scrive* su una pagina condivisa: la MMU genera una trap (protezione in scrittura). Il sistema operativo alloca un nuovo frame, copia il contenuto della pagina originale nel nuovo frame, aggiorna la PTE del processo scrivente in modo che punti al nuovo frame, e infine rimuove il marcatore CoW dalla pagina originale (che ora appartiene solo all'altro processo).

Prima della scrittura



Dopo la scrittura su C (processo 1)



Nella parte superiore del diagramma (prima della scrittura), i due processi condividono i frame fisici A, B e C. Nella parte inferiore (dopo che il processo 1 ha scritto sulla pagina C), il sistema operativo ha allocato un nuovo frame "copia pag. C" per il processo 1. Il processo 2 continua a usare il frame originale C, rimasto invariato.

Osservazione

Il vantaggio della copy-on-write è duplice: la `fork()` è quasi istantanea (si copia solo la tabella delle pagine, non i frame), e la memoria fisica viene duplicata *solo per le pagine effettivamente modificate*, che nella pratica sono una piccola frazione del totale. Nei sistemi in cui `fork()` è immediatamente seguita da `exec()`, quasi nessuna pagina viene mai copiata.

2.11 Impatto realistico del tasso di page fault

Per rendere concreto il ragionamento sulle prestazioni, consideriamo uno scenario tipico.

Esempio

Si supponga che ogni processo abbia 10 pagine ma ne usi attivamente solo 5 per la maggior parte del tempo. Con 40 frame fisici disponibili per i processi utente:

$$\text{numero di processi in esecuzione} = \left\lfloor \frac{40}{5} \right\rfloor = 8$$

Questo permette a 8 processi di girare contemporaneamente, con alta utilizzazione della CPU.
Ora si supponga che il comportamento dei processi cambi improvvisamente e ciascuno abbia bisogno di tutte e 10 le proprie pagine:

$$\text{frame richiesti} = 8 \times 10 = 80 > 40$$

Il sistema non ha abbastanza frame. Ogni rimpiazzamento di una pagina rimuove una pagina che sarà presto ri-richiesta. Si genera una valanga di page fault: il sistema entra in uno stato di **thrashing** (trattato in dettaglio nella sezione dedicata).

Questo esempio mostra come la paginazione su richiesta, pur essendo vantaggiosa nella maggior parte dei casi, possa degenerare in uno stato di crisi quando la domanda di frame supera l'offerta. La gestione di questa situazione richiede meccanismi aggiuntivi: page replacement, controllo del tasso di page fault e, in ultima istanza, riduzione del grado di multiprogrammazione.

3 Sostituzione delle Pagine (Page Replacement)

3.1 Il problema della sovra-allocazione e la necessità del rimpiazzamento

Nella sezione precedente abbiamo visto che il sistema operativo mantiene una lista di frame liberi da assegnare alle pagine richieste. Cosa succede quando questa lista si esaurisce? Il sistema operativo si trova nella situazione in cui deve caricare una nuova pagina dal disco, ma non ha alcun frame disponibile.

Le soluzioni naive sono entrambe inaccettabili:

- **Terminare il processo:** interrompe un processo legittimo per una ragione che il programmatore non può controllare. Non è trasparente e viola il contratto implicito della memoria virtuale.
- **Fare swapping dell'intero processo:** scrivere tutte le pagine di un processo sul disco e liberarne i frame è un'operazione estremamente costosa in termini di I/O.

La soluzione moderna è la **sostituzione delle pagine** (*page replacement*): il sistema operativo sceglie una pagina già in RAM — chiamata **pagina vittima** (*victim page*) — la rimuove dalla memoria (scrivendola su disco se è stata modificata) e utilizza il frame liberato per la pagina richiesta.

Definizione

La **sostituzione delle pagine** è il meccanismo con cui il sistema operativo libera un frame fisico occupato da una pagina non più strettamente necessaria, al fine di caricarvi una pagina richiesta da un page fault quando non esistono frame liberi. La scelta della pagina vittima è determinata dall'**algoritmo di sostituzione delle pagine**.

La sequenza completa di un page fault con sostituzione è:

1. Si verifica un page fault su una pagina P.
2. Non esistono frame liberi.
3. L'algoritmo di sostituzione seleziona una pagina vittima V attualmente in RAM.
4. Se V è stata modificata (*dirty*), viene scritta sul disco (*page out*).
5. La pagina P viene letta dal disco nel frame liberato (*page in*).
6. La PTE di V viene aggiornata (bit non valido); la PTE di P viene aggiornata (bit valido, numero del frame).
7. L'istruzione interrotta viene riavviata.

Attenzione

Quando è necessaria sia una scrittura che una lettura su disco (la vittima è dirty e la nuova pagina deve essere caricata), il tempo di servizio del page fault *raddoppia*, poiché si effettuano due operazioni di I/O invece di una. Questo ha un impatto diretto sull'EAT e sottolinea l'importanza del **bit di modifica**.

3.2 Il bit di modifica (dirty bit)

Per evitare di scrivere su disco pagine che non sono state modificate — operazione inutile e costosa — ogni PTE contiene un **bit di modifica** (*dirty bit* o *modify bit*).

Definizione

Il **dirty bit** di una PTE è un flag impostato automaticamente dall'hardware (MMU) ogni volta che il processo *scrive* sulla pagina corrispondente. Se il dirty bit è `1`, la pagina contiene dati più recenti di quelli presenti su disco e deve essere salvata prima di essere rimossa dalla RAM. Se il dirty bit è `0`, la copia su disco è ancora valida e la pagina può essere semplicemente scartata senza alcuna scrittura.

Le pagine di codice (istruzioni) sono tipicamente di sola lettura: il loro dirty bit è sempre `0` e possono sempre essere scartate senza scrittura, potendo essere ricaricate direttamente dal file eseguibile. Le pagine di dati e dello stack, invece, vengono spesso modificate e richiedono scrittura su disco prima della sostituzione.

Osservazione

Il dirty bit è uno degli esempi più eleganti di collaborazione tra hardware e sistema operativo: l'hardware lo imposta automaticamente ad ogni scrittura (senza alcun overhead per il processo), e il sistema operativo lo consulta solo quando deve scegliere una vittima, risparmiando potenzialmente metà delle operazioni di I/O durante il rimpiazzamento.

3.3 Ruolo della sostituzione nella memoria virtuale

La sostituzione delle pagine completa il meccanismo della memoria virtuale: grazie ad essa, lo spazio di indirizzamento logico di un processo può superare la dimensione della memoria fisica senza che il processo ne sia consapevole.

Esempio

Un programma con 20 pagine logiche può girare su un sistema con soli 10 frame fisici. In ogni momento, al massimo 10 pagine sono in RAM; le altre 10 sono su disco. Quando il programma accede a una pagina non in RAM, il sistema operativo rimpiazza una delle 10 pagine presenti e carica quella richiesta. Dal punto di vista del programma, la memoria appare sempre disponibile e contigua.

3.4 Valutazione degli algoritmi: la stringa di riferimento

Per confrontare le prestazioni dei diversi algoritmi di sostituzione è necessario un metodo di valutazione oggettivo. Il numero di page fault generati da un algoritmo dipende dalla sequenza degli accessi alla memoria del processo — la **stringa di riferimento** — e dal numero di frame disponibili.

Definizione

La **stringa di riferimento** (*reference string*) è la sequenza ordinata dei numeri di pagina acceduti da un processo durante la sua esecuzione. Per costruirla si osserva la sequenza degli indirizzi logici generati dalla CPU e si conserva solo il numero di pagina (dividendo l'indirizzo per la dimensione della pagina), eliminando gli accessi consecutivi alla stessa pagina (poiché, una volta caricata, una pagina rimane in RAM per accessi ripetuti senza generare nuovi fault).

Esempio

Si considerino gli indirizzi (in byte) con pagine da 100 byte:

$$0100,\ 0432,\ 0101,\ 0612,\ 0102,\ 0613,\ 0103,\ 0614,\ 0104,\ 0615,\ 0105$$

Dividendo per 100 si ottengono i numeri di pagina:

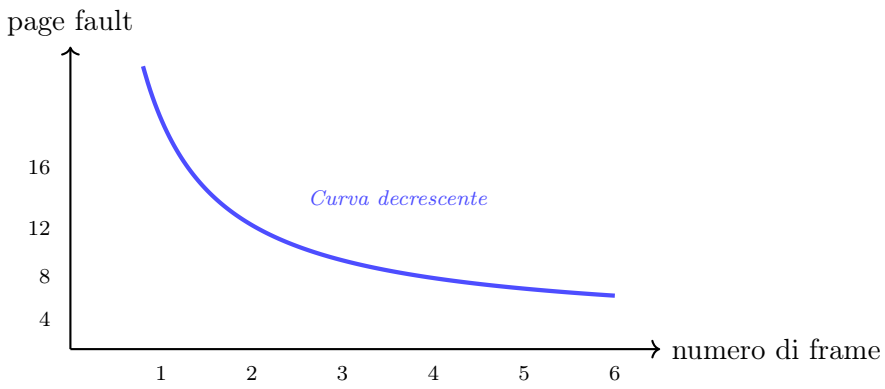
$$1,\ 4,\ 1,\ 6,\ 1,\ 6,\ 1,\ 6,\ 1,\ 6,\ 1$$

Poiché accessi consecutivi alla stessa pagina non generano fault aggiuntivi, la stringa di riferimento effettiva è:

$$1,\ 4,\ 1,\ 6,\ 1,\ 6,\ 1,\ 6,\ 1,\ 6,\ 1$$

Ci sono 3 pagine distinte (1, 4, 6). Con 3 o più frame disponibili si verificano solo 3 page fault (uno per pagina, al primo accesso). Con 1 solo frame si verificano 11 page fault (uno per ogni accesso, poiché ogni nuovo accesso espelle la pagina corrente).

Effetto del numero di frame. In generale, all'aumentare del numero di frame disponibili il numero di page fault tende a diminuire, fino a raggiungere un minimo quando tutti i frame necessari sono disponibili simultaneamente. La curva page fault / frame ha tipicamente un andamento decrescente e convesso, come mostrato qualitativamente di seguito:



La curva mostra che i guadagni maggiori si ottengono aggiungendo i primi frame; oltre un certo numero il beneficio marginale si riduce rapidamente.

3.5 Algoritmo FIFO (First-In, First-Out)

Il primo algoritmo di sostituzione che esaminiamo è anche il più intuitivo.

Definizione

L'**algoritmo FIFO** (*First-In, First-Out*) sostituisce sempre la pagina che si trova in memoria da più tempo, ovvero la pagina *più vecchia*. Viene implementato con una coda: le pagine vengono inserite in coda all'arrivo e la vittima è sempre quella in testa (la prima entrata).

La logica intuitiva è che una pagina caricata da molto tempo sia meno utile di una caricata di recente. Come vedremo, questa ipotesi è spesso sbagliata.

Esempio

Stringa di riferimento:

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2,\ 1,\ 2,\ 0,\ 1,\ 7,\ 0,\ 1$$

I primi 3 accessi (7, 0, 1) caricano le prime 3 pagine nei 3 frame disponibili (tutti e 3 generano page fault, poiché i frame sono inizialmente vuoti). Da quel momento, ogni nuovo accesso a una pagina non presente in memoria genera un fault e la vittima è la pagina più vecchia in coda.
Svolgendo la simulazione passo per passo si contano **15 page fault** su 20 riferimenti.

L'algoritmo FIFO è semplice da implementare (basta una coda circolare) e non richiede supporto hardware speciale. Tuttavia presenta un grave difetto: può espellere pagine molto usate solo perché sono in memoria da tanto tempo.

3.6 L'anomalia di Bélády

Un risultato sorprendente riguarda il comportamento di FIFO all'aumentare del numero di frame.

Definizione

L'**anomalia di Bélády** è il fenomeno per cui, con l'algoritmo FIFO, *aumentare il numero di frame disponibili può aumentare il numero di page fault* invece di diminuirlo. Questo comportamento controintuitivo non si verifica con gli algoritmi ottimali basati sulla struttura degli accessi.

Esempio

Con la stringa di riferimento:

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5$$

- Con **3 frame**: 9 page fault.
- Con **4 frame**: 10 page fault.

Passando da 3 a 4 frame il numero di page fault *aumenta* di 1. Il grafico dell'anomalia mostra un andamento non monotonicamente decrescente, con un picco a 4 frame.

Osservazione

L'anomalia di Bélády nasce dal fatto che FIFO non tiene conto della *frequenza di utilizzo* delle pagine, ma solo della loro anzianità in memoria. Aggiungere un frame cambia quali pagine vengono espulse e in quale ordine, e può portare a espellere pagine che saranno richieste molto presto.
Gli algoritmi che godono della proprietà **stack property** — ovvero che l'insieme di pagine in memoria con $n+1$ frame è sempre un soprainsieme di quello con n frame — sono immuni all'anomalia di Bélády. LRU e OPT godono di questa proprietà; FIFO no.

3.7 Algoritmo Ottimale (OPT / MIN)

Definizione

L'**algoritmo ottimale** (detto anche OPT o MIN) sostituisce sempre la pagina il cui **prossimo utilizzo è più lontano nel futuro**. Tra tutte le pagine attualmente in memoria, viene scelta come vittima quella che non verrà usata per il periodo più lungo (o che non verrà mai più usata).

Proprietà 3.1. *L'algoritmo OPT genera il* ***numero minimo possibile di page fault*** *tra tutti gli algoritmi di sostituzione, per qualsiasi stringa di riferimento e qualsiasi numero di frame. È immune all'anomalia di Bélády.*

Esempio

Con la stessa stringa di riferimento dell'esempio FIFO:

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2,\ 1,\ 2,\ 0,\ 1,\ 7,\ 0,\ 1$$

OPT genera soli **9 page fault**, contro i 15 di FIFO.

Attenzione

L'algoritmo OPT è **non implementabile** in un sistema operativo reale: richiederebbe la conoscenza degli accessi futuri, che non sono disponibili a priori. OPT viene usato esclusivamente come **benchmark teorico**: confrontando le prestazioni di un algoritmo reale con OPT si ottiene una misura di quanto ci si avvicina al comportamento ideale.

3.8 Algoritmo LRU (Least Recently Used)

L'algoritmo LRU è l'approssimazione pratica più usata dell'algoritmo ottimale. Si basa sull'osservazione che il passato recente è un buon predittore del futuro prossimo.

Definizione

L'**algoritmo LRU** (*Least Recently Used*) sostituisce la pagina che **non è stata usata per il periodo più lungo**, ovvero quella il cui ultimo accesso risale al momento più remoto nel passato. LRU usa la storia degli accessi passati come approssimazione degli accessi futuri.

La simmetria con OPT è precisa: OPT guarda avanti nel tempo e sceglie la pagina usata più tardi nel futuro; LRU guarda indietro e sceglie la pagina usata più indietro nel passato.

Esempio

Con la stessa stringa di riferimento:

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2,\ 1,\ 2,\ 0,\ 1,\ 7,\ 0,\ 1$$

LRU genera **12 page fault**: peggio di OPT (9) ma significativamente meglio di FIFO (15).

LRU gode della *stack property* ed è quindi immune all'anomalia di Bélády.

3.9 Implementazione di LRU

LRU richiede di conoscere, per ogni pagina in memoria, il momento del suo ultimo accesso. Esistono due approcci principali.

Implementazione con contatore (*counter-based*). La CPU mantiene un **contatore globale** che viene incrementato ad ogni riferimento alla memoria. Ogni PTE contiene un campo *timestamp* che viene aggiornato con il valore corrente del contatore ogni volta che la pagina viene acceduta. Per scegliere la vittima si scansiona la tabella delle pagine e si seleziona la PTE con il timestamp più piccolo (accesso più remoto nel tempo).

- **Vantaggio:** semplice da capire.
- **Svantaggio:** richiede una scrittura in memoria ad ogni accesso (per aggiornare il timestamp) e una scansione lineare della tabella al momento della sostituzione. Entrambe le operazioni sono costose.

Implementazione con stack (*stack-based*). Si mantiene uno **stack di numeri di pagina** ordinato per recenza di accesso: la pagina in cima è quella acceduta più di recente, quella in fondo è la LRU (la vittima).

Ad ogni accesso alla pagina p:

1. Si rimuove p dalla sua posizione attuale nello stack (se presente).
2. Si inserisce p in cima allo stack.

La vittima è sempre la pagina in fondo, accessibile in tempo costante.

- **Vantaggio:** non richiede scansione per trovare la vittima.
- **Svantaggio:** ogni accesso richiede un aggiornamento della struttura dello stack (rimozione e reinserimento), che su hardware è costoso da implementare efficientemente.

Il diagramma seguente mostra l'evoluzione dello stack durante la stringa 4, 7, 0, 7, 1, 0, 1, 2, 1, 2, 7, dopo il riferimento **a** (accesso a pagina 2) lo stack è $[2, 1, 0, 7, 4]$ (cima = più recente); dopo il riferimento **b** (accesso a pagina 7) lo stack diventa $[7, 2, 1, 0, 4]$. La pagina in fondo (4) è la LRU in entrambi i casi.

Nota

Nessuna delle due implementazioni è praticabile in hardware puro senza costi proibitivi. Per questo motivo i sistemi reali utilizzano *approssimazioni* di LRU, descritte nelle sottosezioni seguenti.

3.10 Approssimazioni di LRU: il bit di riferimento

La maggior parte dei processori moderni non implementa LRU esatto in hardware. Tuttavia fornisce un supporto minimo tramite il **bit di riferimento** (*reference bit*).

Definizione

Il **bit di riferimento** è un flag presente in ogni PTE, impostato automaticamente dall'hardware (MMU) ogni volta che la pagina corrispondente viene *acceduta* (in lettura o in scrittura). Il sistema operativo può leggere e azzerare questo bit periodicamente. Un bit a `1` indica che la pagina è stata usata di recente; un bit a `0` indica che non è stata usata dall'ultimo azzeramento.

Il bit di riferimento da solo non basta per implementare LRU completo: indica *se* una pagina è stata usata nell'intervallo, non *quando*. Tuttavia costituisce la base per diverse approssimazioni pratiche.

3.11 LRU approssimato: registro di storia degli accessi

Un'approssimazione più accurata usa un **registro a scorrimento** (*shift register*) di k bit per ogni pagina, mantenuto dal sistema operativo.

Il meccanismo funziona come segue:

- Un'interruzione del timer scatta ogni intervallo fisso (es. ogni 100 ms).
- Ad ogni interruzione, per ogni pagina:
  1. Il valore corrente del bit di riferimento viene inserito come bit più significativo del registro di storia.
  2. L'intero registro viene fatto scorrere di 1 bit verso destra (il bit meno significativo viene perso).
  3. Il bit di riferimento viene azzerato.
- Il registro a 8 bit di ogni pagina rappresenta uno "storico" degli ultimi 8 intervalli temporali.

Interpretazione del registro.

- `00000000`: la pagina non è stata usata in nessuno degli ultimi 8 intervalli → ottima candidata alla sostituzione.
- `11111111`: la pagina è stata usata in tutti gli ultimi 8 intervalli → pessima candidata.
- `01000000`: la pagina è stata usata 2 intervalli fa ma non più di recente.
- `00000001`: la pagina è stata usata 8 intervalli fa ma non più di recente.

La pagina con il **valore binario più piccolo** del registro viene scelta come vittima (uso più remoto). In caso di parità si usa FIFO o un'altra euristica.

3.12 Algoritmo della Seconda Chance (Second-Chance)

Definizione

L'**algoritmo della seconda chance** è una variante di FIFO che usa il bit di riferimento per evitare di espellere pagine usate di recente. Prima di rimuovere la pagina più vecchia in coda, si controlla il suo bit di riferimento:

- **Bit = 0:** la pagina non è stata usata di recente → viene sostituita (come in FIFO).
- **Bit = 1:** la pagina ha una "seconda chance": il bit viene azzerato, la pagina viene spostata in fondo alla coda (come se fosse appena arrivata), e si passa alla pagina successiva.

Una pagina con bit di riferimento sempre a `1` può rimanere in memoria indefinitamente, venendo continuamente "graziata" e rimessa in fondo alla coda.

3.13 Algoritmo Clock

L'algoritmo Clock è l'implementazione circolare dell'algoritmo della seconda chance, più efficiente dal punto di vista pratico.

Definizione

Nell'**algoritmo Clock** le pagine sono organizzate in una **coda circolare**. Un puntatore (*lancetta dell'orologio*) scansiona le pagine in ordine circolare. Quando occorre una sostituzione:

- Se la pagina puntata ha bit di riferimento = `1`: azzera il bit e avanza la lancetta.
- Se la pagina puntata ha bit di riferimento = `0`: sostituisci questa pagina e avanza la lancetta.

Nel caso peggiore (tutti i bit = `1`) la lancetta compie un giro completo azzerando tutti i bit prima di trovare una vittima, comportandosi come FIFO. Nel caso medio la lancetta si ferma dopo pochi passi.

Il nome "Clock" deriva dalla metafora del quadrante di un orologio: le pagine sono disposte come i numeri sul quadrante, e la lancetta ruota cercando una vittima.

3.14 Seconda chance migliorata: bit di riferimento e bit di modifica

Un'ulteriore raffinazione dell'algoritmo Clock usa sia il bit di riferimento che il dirty bit, classificando le pagine in quattro categorie ordinate per preferenza di sostituzione:

Ref	Dirty	Classe	Significato
0	0	(0,0)	Non usata di recente, non modificata: **miglior candidata** — nessuna scrittura su disco necessaria.
0	1	(0,1)	Non usata di recente, modificata: da scrivere su disco prima di espellere.
1	0	(1,0)	Usata di recente, non modificata: probabilmente sarà richiesta presto.
1	1	(1,1)	Usata di recente, modificata: **peggior candidata** — costosa da espellere e presto ri-richiesta.

L'algoritmo scansiona le pagine in stile Clock e seleziona la prima pagina appartenente alla classe non vuota con numero più basso. Se non trova candidati nelle classi (0,0) e (0,1), effettua un secondo giro dopo aver azzerato i bit di riferimento.

Osservazione

Questa classificazione ottimizza simultaneamente due obiettivi: minimizzare le scritture su disco (preferire pagine con dirty bit = 0) e minimizzare i page fault futuri (preferire pagine con bit di riferimento = 0). È una delle approssimazioni di LRU più usate nella pratica.

3.15 Algoritmi LFU e MFU

Esistono altri algoritmi di sostituzione basati sulla *frequenza* di accesso piuttosto che sulla recenza.

LFU (Least Frequently Used). Sostituisce la pagina con il **contatore di accessi più basso**. L'ipotesi è che una pagina poco usata in passato sarà poco usata anche in futuro.

- **Problema:** una pagina usata intensamente nelle fasi iniziali del programma ma non più necessaria mantiene un contatore alto e non viene mai espulsa, sprecando un frame.
- **Possibile soluzione:** ridurre periodicamente tutti i contatori (es. shift right di 1 bit) per far "invecchiare" la storia passata.

MFU (Most Frequently Used). Sostituisce la pagina con il **contatore di accessi più alto**. La logica è che una pagina molto acceduta in passato potrebbe avere appena terminato la sua utilità.

Nota

Né LFU né MFU approssimano bene l'algoritmo OPT e non godono della *stack property*. Nella pratica sono raramente usati da soli; possono tuttavia essere utili in combinazione con altri meccanismi in contesti specifici (es. buffer cache dei file system).

3.16 Strategie di supporto alla sostituzione

Per migliorare l'efficienza della sostituzione, i sistemi operativi adottano alcune strategie ausiliarie.

Pool di frame liberi. Invece di cercare una vittima *solo* quando si verifica un page fault, il sistema operativo mantiene sempre una piccola riserva di frame liberi (il *free-frame pool*). Quando si verifica un page fault, la nuova pagina viene caricata immediatamente in un frame del pool, e il processo riprende l'esecuzione *senza attendere* la scrittura su disco della vittima (che avviene in background). Questo riduce la latenza percepita dal processo.

Lista delle pagine modificate. Il sistema operativo mantiene una lista delle pagine con dirty bit = `1`. Quando il dispositivo di paging è inattivo (es. durante i periodi di bassa attività), le pagine in questa lista vengono scritte su disco in anticipo e il loro dirty bit viene azzerato. In questo modo, quando vengono scelte come vittime, non richiedono alcuna scrittura, dimezzando il tempo di servizio del page fault.

Riutilizzo dei frame liberi. Quando una pagina viene rimossa dalla RAM e il suo frame viene inserito nel pool dei liberi, il contenuto del frame non viene immediatamente cancellato. Se la stessa pagina viene ri-richiesta *prima* che il frame venga riutilizzato, il sistema operativo può soddisfare il page fault senza alcuna lettura da disco, semplicemente "recuperando" il frame dal pool e aggiornando la PTE. Solo quando il frame viene effettivamente riassegnato a un'altra pagina il contenuto precedente viene perso.

4 Allocazione dei Frame

4.1 Il problema centrale dell'allocazione

Fino a questo punto abbiamo discusso *come* sostituire una pagina quando non ci sono frame liberi. Rimane però aperta una domanda altrettanto importante: *quanti frame assegnare a ciascun processo?* Questa decisione ha conseguenze dirette sul tasso di page fault di ogni processo e sull'efficienza globale del sistema.

Definizione

Il **problema dell'allocazione dei frame** consiste nel decidere quanti frame fisici assegnare a ciascun processo attivo, in modo da bilanciare equità, efficienza e prevenzione del thrashing. La decisione deve essere presa sia staticamente (all'avvio del processo) che dinamicamente (durante l'esecuzione, al variare del carico del sistema).

Esempio

Si consideri un sistema con 128 frame totali. Il sistema operativo ne riserva 35 per sé (kernel, buffer di I/O, strutture dati interne). Rimangono 93 frame per i processi utente.
Con la paginazione su richiesta pura, all'avvio tutti i 93 frame entrano nella lista dei frame liberi. I primi 93 page fault di un processo vengono soddisfatti semplicemente prelevando frame dalla lista. Al 94° page fault non esistono più frame liberi e il sistema deve ricorrere alla sostituzione.
Quando il processo termina, tutti i suoi frame vengono restituiti alla lista dei liberi, pronti per il prossimo processo.

4.2 Vincoli e requisiti minimi

L'allocazione dei frame non è arbitraria: deve rispettare due vincoli fondamentali.

Vincolo superiore. Un processo non può ricevere più frame di quanti ne esistano fisicamente (a meno che non si usi condivisione di pagine, che è un caso speciale). Formalmente, se m è il numero totale di frame disponibili per i processi utente e n è il numero di processi, l'allocazione a_i del processo i deve soddisfare:

$$\sum_{i=1}^{n} a_i \leq m$$

Vincolo inferiore: il minimo garantito. Ogni processo deve ricevere un **numero minimo di frame** sufficiente a contenere tutte le pagine che una singola istruzione può referenziare simultaneamente. Se un processo ha meno frame del minimo, ogni istruzione può generare un page fault, il che porta a page fault ripetuti sulla stessa istruzione senza possibilità di completarla.

Osservazione

Il numero minimo di frame per processo dipende dall'**architettura hardware**. Su un'architettura semplice (es. accesso a memoria con indirizzo diretto) bastano 2 frame (uno per il codice, uno per i dati). Su architetture più complesse, dove una singola istruzione può referenziare indirettamente più locazioni di memoria (es. istruzioni con modalità di indirizzamento indiretto a più livelli), il minimo può essere di 6–8 frame o più. Il **massimo** di frame per processo è invece limitato solo dalla RAM totale disponibile.

4.3 Allocazione uniforme

Definizione

L'**allocazione uniforme** (*equal allocation*) distribuisce i frame disponibili in parti uguali tra tutti i processi. Se ci sono m frame e n processi, ogni processo riceve:

$$a_i = \left\lfloor \frac{m}{n} \right\rfloor \quad \text{frame}$$

I frame rimanenti ($(m \bmod n)$) vengono tenuti nel pool dei liberi o assegnati secondo criteri secondari.

Esempio

Con 93 frame e 5 processi:

$$a_i = \left\lfloor \frac{93}{5} \right\rfloor = 18 \quad \text{frame per processo}$$

Rimangono $93 - 5 \times 18 = 3$ frame nel pool dei liberi.

Limite dell'allocazione uniforme. L'allocazione uniforme ignora completamente le dimensioni e le esigenze dei processi. Un processo che usa 10 pagine e uno che ne usa 10 000 ricevono lo stesso numero di frame. Il processo piccolo avrà frame in eccesso (sprecati), mentre quello grande soffrirà di un tasso di page fault elevato.

4.4 Allocazione proporzionale

L'allocazione proporzionale corregge il difetto dell'allocazione uniforme tenendo conto della dimensione di ciascun processo.

Definizione

L'**allocazione proporzionale** (*proportional allocation*) assegna a ciascun processo un numero di frame proporzionale alla sua dimensione (numero di pagine virtuali). Formalmente, siano s_i il numero di pagine del processo p_i, $S = \sum_i s_i$ la domanda totale di pagine e m il numero totale di frame disponibili. L'allocazione del processo p_i è:

$$a_i = \frac{s_i}{S} \times m$$

Il risultato viene arrotondato all'intero più vicino e verificato rispetto al minimo architetturale, con il vincolo $\sum_i a_i \leq m$.

Esempio

Si abbiano 62 frame disponibili e due processi:

- p_1: 10 pagine ($s_1 = 10$)
- p_2: 127 pagine ($s_2 = 127$)

$S = 10 + 127 = 137$. Le allocazioni sono:

$$a_1 = \frac{10}{137} \times 62 \approx 4{,}5 \to 5 \quad \text{frame}$$

$$a_2 = \frac{127}{137} \times 62 \approx 57{,}5 \to 57 \quad \text{frame}$$

Totale assegnato: $5 + 57 = 62 \leq m$. ✓
Si noti che un'allocazione uniforme avrebbe assegnato 31 frame a ciascuno: troppi per p_1 (che ne usa solo 10) e troppo pochi per p_2 (che ne usa 127).

Allocazione proporzionale con priorità. L'allocazione proporzionale può essere estesa per tenere conto della **priorità** dei processi oltre che della loro dimensione. Un processo ad alta priorità riceve una quota di frame maggiore rispetto a quanto suggerirebbe la sola dimensione. In sistemi con priorità, si può usare una formula del tipo:

$$a_i = \frac{w_i}{\sum_j w_j} \times m$$

dove w_i è un peso che combina dimensione e priorità del processo i.

Osservazione

L'allocazione dei frame è dinamica: al variare del grado di multiprogrammazione (nuovi processi che entrano, processi che terminano), le allocazioni di tutti i processi vengono ricalcolate. Un processo che termina libera i suoi frame, che vengono redistribuiti agli altri.

4.5 Sostituzione locale e sostituzione globale

Una volta stabilito il numero di frame da assegnare a ciascun processo, occorre definire come avviene la sostituzione: un processo può attingere solo ai propri frame o anche a quelli degli altri?

Definizione

Nella **sostituzione locale** (*local replacement*), quando un processo deve rimpiazzare una pagina, può scegliere la vittima *solo tra i propri frame*. Il numero di frame assegnato al processo rimane costante nel tempo.
Nella **sostituzione globale** (*global replacement*), il processo vittima può appartenere a *qualsiasi* processo in memoria, inclusi altri processi con priorità inferiore. Il numero di frame di ciascun processo varia dinamicamente.

Confronto tra le due politiche.

	Sostituzione locale	**Sostituzione globale**
Tasso di page fault	Prevedibile: dipende solo dal comportamento del processo stesso.	Imprevedibile: può essere influenzato dal comportamento di altri processi.
Throughput	Potenzialmente inferiore: frame inutilizzati da un processo non possono essere usati da altri.	Generalmente superiore: i frame vengono usati dove servono di più.
Isolamento	Elevato: un processo non può degradare le prestazioni di un altro.	Basso: un processo "aggressivo" può sottrarre frame agli altri.
Uso in pratica	Meno comune nei SO moderni.	Predominante (Linux, Windows, macOS).

Nota

I sistemi con priorità possono adottare una politica ibrida: la sostituzione globale è consentita, ma un processo ad alta priorità può sottrarre frame solo a processi a priorità *inferiore*, non a processi con priorità uguale o superiore.

4.6 Soglie di frame liberi nella sostituzione globale

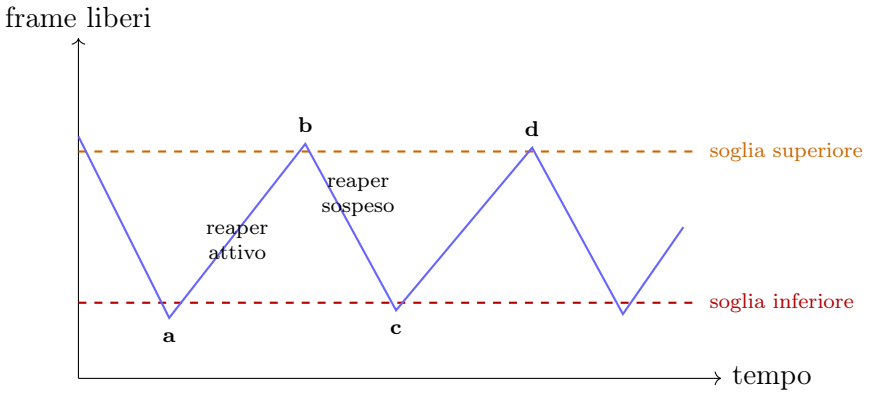
Nei sistemi con sostituzione globale, il sistema operativo deve assicurarsi che ci sia sempre un certo numero minimo di frame liberi disponibili per soddisfare rapidamente i page fault. A questo scopo si usa un meccanismo a soglie.

Definizione

Il sistema operativo definisce due soglie per la memoria libera:

- **Soglia inferiore** (*minimum threshold*): se il numero di frame liberi scende al di sotto di questo valore, il sistema operativo attiva un processo di recupero pagine (*reaper* o *kswapd* in Linux) che inizia a espellere pagine dalla RAM.
- **Soglia superiore** (*maximum threshold*): il processo di recupero continua a liberare frame fino a quando il numero di frame liberi supera questo valore, poi si sospende.

Il processo di recupero opera in background, in modo che quando si verifica un page fault esista già un frame libero pronto, senza che il processo richiedente debba attendere operazioni di I/O per liberarne uno. Il grafico seguente mostra l'andamento oscillante della memoria libera nel tempo: scende al di sotto della soglia inferiore (punto **a**), il reaper interviene portandola sopra la soglia superiore (punto **b**), poi si sospende e la memoria libera ricomincia a scendere (punto **c**), e il ciclo si ripete.



4.7 Architetture NUMA e allocazione dei frame

Nei sistemi moderni con più processori, la memoria fisica non è sempre equidistante da tutti i core. Questa caratteristica ha implicazioni dirette sull'allocazione dei frame.

Definizione

Un'**architettura NUMA** (*Non-Uniform Memory Access*) è un sistema multiprocessore in cui ogni CPU (o gruppo di CPU) ha una propria memoria *locale* con latenza di accesso ridotta. L'accesso alla memoria *remota* (quella di un altro nodo NUMA) è significativamente più lento, tipicamente di un fattore 2–5 rispetto all'accesso locale.

In un sistema NUMA, l'allocazione dei frame deve tenere conto della **prossimità fisica** tra la CPU che esegue il processo e i frame assegnati a quel processo.

- Quando si verifica un page fault, il sistema operativo tenta di allocare un frame nella memoria *locale* alla CPU su cui è in esecuzione il processo.
- Lo scheduler tiene traccia dell'ultima CPU usata da ciascun processo, in modo da minimizzare gli accessi remoti.
- Allocare frame locali migliora la *cache locality*: i dati acceduti frequentemente rimangono nei livelli di cache della CPU locale, riducendo ulteriormente la latenza.

Osservazione

Nei sistemi NUMA la scelta "sbagliata" del nodo di allocazione può degradare significativamente le prestazioni anche in assenza di page fault. Un processo che accede continuamente a frame remoti paga un costo di latenza ad ogni accesso, indipendentemente dal fatto che la pagina sia in RAM. Questo è un esempio di come la gestione della memoria virtuale sia strettamente intrecciata con l'architettura hardware sottostante.

5 Thrashing e Modelli di Controllo

5.1 Definizione e cause del thrashing

Abbiamo visto che la paginazione su richiesta permette di eseguire più processi di quanti ne potrebbero stare interamente in RAM, grazie al fatto che ogni processo usa attivamente solo una parte delle proprie pagine. Tuttavia questa strategia nasconde un rischio grave: se il sistema assegna a un processo troppo pochi frame rispetto alle sue reali necessità, il processo entra in uno stato patologico chiamato **thrashing**.

Definizione

Il **thrashing** è la condizione in cui un processo (o l'intero sistema) trascorre più tempo a gestire page fault e operazioni di I/O su disco che a eseguire istruzioni utili. Si verifica quando il numero di frame assegnati a un processo è inferiore al numero di pagine che esso usa attivamente: ogni sostituzione rimuove una pagina che sarà presto ri-richiesta, generando immediatamente un nuovo page fault.

La catena causale che porta al thrashing in un sistema con sostituzione globale è la seguente:

1. Un processo ha bisogno di più frame di quanti ne abbia assegnati.
2. Genera frequenti page fault e sottrae frame ad altri processi (sostituzione globale).
3. I processi che hanno perso frame generano a loro volta page fault e sottraggono frame ad altri ancora.
4. Il dispositivo di paging (disco) viene saturato da richieste di I/O concorrenti.
5. La coda dei processi pronti si svuota: i processi sono tutti in attesa di I/O.
6. La CPU risulta quasi inutilizzata nonostante il sistema sia apparentemente sovraccarico.

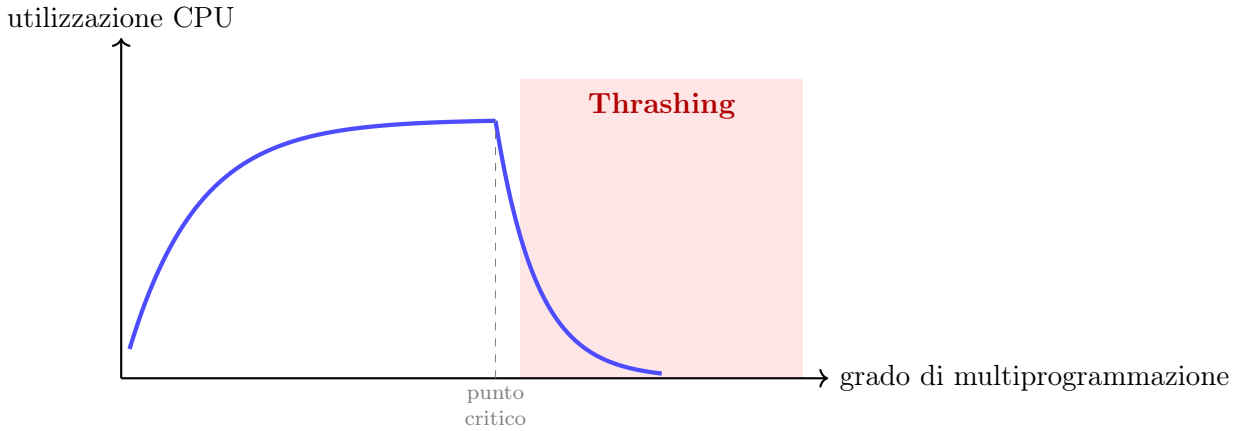
Attenzione

Il thrashing è una trappola insidiosa per il sistema operativo: la bassa utilizzazione della CPU sembra suggerire che si possano aggiungere altri processi per aumentare il throughput. Ma l'aggiunta di nuovi processi peggiora ulteriormente la situazione, sottraendo ancora più frame a quelli esistenti e amplificando il ciclo vizioso.

5.2 Thrashing e utilizzazione della CPU

Il rapporto tra grado di multiprogrammazione e utilizzazione della CPU ha un andamento caratteristico che illustra chiaramente il fenomeno del thrashing.

All'aumentare del numero di processi in esecuzione simultanea, l'utilizzazione della CPU cresce: più processi significano più lavoro da fare, e quando un processo è in attesa di I/O un altro può usare la CPU. Tuttavia, oltre un certo punto critico, l'aggiunta di nuovi processi causa un crollo improvviso dell'utilizzazione: il sistema entra in thrashing.



Il grafico mostra che esiste un **punto critico** oltre il quale aumentare ulteriormente il grado di multiprogrammazione provoca un crollo dell'utilizzazione della CPU. La soluzione è **ridurre il grado di multiprogrammazione**: sospendere uno o più processi, scriverne le pagine su disco e redistribuire i frame liberati ai processi rimanenti. Questo permette ai processi attivi di avere abbastanza frame per operare senza thrashing.

5.3 Soluzione locale vs globale al thrashing

La sostituzione locale limita il danno del thrashing: un processo in thrashing non può sottrarre frame agli altri, quindi il degrado rimane confinato al processo stesso. Tuttavia, anche con sostituzione locale, un processo in thrashing satura il dispositivo di paging, aumentando i tempi di servizio dei page fault per *tutti* i processi, anche quelli non in thrashing.

La soluzione definitiva al thrashing richiede di garantire a ciascun processo abbastanza frame da contenere la sua **località corrente**. Per fare ciò occorre un modello che stimi quanti frame un processo necessita in ogni momento della sua esecuzione.

5.4 Il modello della località

Definizione

Una **località** di un processo è un insieme di pagine che il processo usa attivamente e frequentemente in un determinato intervallo di tempo. Durante la sua esecuzione, un processo si sposta da una località all'altra: quando chiama una procedura, entra in una nuova località (il codice della procedura, le sue variabili locali, alcune variabili globali); quando la procedura termina, esce da quella località. Località diverse possono sovrapporsi parzialmente.

Il concetto di località giustifica la paginazione su richiesta: se un processo usa attivamente solo un sottoinsieme limitato delle sue pagine in ogni momento, è sufficiente tenere in RAM solo quel sottoinsieme. Il thrashing si verifica quando il numero di frame è inferiore alla dimensione della località corrente.

Osservazione

L'esistenza delle località non è accidentale: è una conseguenza diretta della struttura dei programmi. I cicli accedono ripetutamente alle stesse istruzioni e variabili (località temporale); le istruzioni di un ciclo e i dati su cui opera occupano pagine vicine (località spaziale). I sistemi di cache hardware sfruttano la stessa proprietà.
Il grafico degli accessi alla memoria di un processo nel tempo mostra chiaramente questo comportamento: densi cluster di riferimenti alle stesse pagine (le località), separati da transizioni rapide verso nuovi cluster quando il processo cambia fase di esecuzione.

5.5 Il modello del working set

Il **modello del working set** è il principale strumento teorico e pratico per stimare quanti frame un processo necessita in un dato momento.

Definizione

Il **working set** di un processo al tempo t, con finestra di dimensione Δ, è l'insieme delle pagine referenziate dal processo nelle ultime Δ referenze alla memoria:

$$WS(t, \Delta) = \{\text{pagine referenziate nell'intervallo } (t - \Delta,\ t]\}$$

La **dimensione del working set** $\mathrm{WSS}(t)$ è la cardinalità di questo insieme: $\mathrm{WSS}(t) = |WS(t, \Delta)|$.

Il parametro Δ è la **finestra del working set**:

- Se Δ è troppo piccolo, il working set non cattura l'intera località corrente del processo, sottostimando le sue necessità.
- Se Δ è troppo grande, il working set può includere pagine di più località distinte, sovrastimando le necessità.
- Il valore ottimale di Δ è quello che approssima meglio la dimensione della località corrente.

Esempio

Si consideri la seguente stringa di riferimento e $\Delta = 10$:

$$\ldots\ 2\ 6\ 1\ 5\ 7\ 7\ 7\ 7\ 5\ 1\ 6\ 2\ 3\ 4\ 1\ 2\ 3\ 4\ 4\ 4\ 3\ 4\ 3\ 4\ 4\ 4\ 1\ 3\ 2\ 3\ 4\ 4\ 4\ 3\ 4\ 4\ 4\ \ldots$$

- Al tempo t_1: le ultime 10 referenze accedono alle pagine $\{1, 2, 5, 6, 7\} \Rightarrow WS(t_1) = \{1, 2, 5, 6, 7\}$, $\mathrm{WSS}(t_1) = 5$.
- Al tempo t_2 (più avanti nell'esecuzione): le ultime 10 referenze accedono alle pagine $\{3, 4\} \Rightarrow WS(t_2) = \{3, 4\}$, $\mathrm{WSS}(t_2) = 2$.

Il processo ha cambiato località: da una che richiedeva 5 pagine a una che ne richiede solo 2.

Uso del working set per prevenire il thrashing. Il sistema operativo calcola la domanda totale di frame come:

$$D = \sum_{i=1}^{n} \mathrm{WSS}_i$$

dove la somma è estesa a tutti gli n processi attivi. Se $D > m$ (la domanda supera i frame disponibili), il sistema è in pericolo di thrashing. La soluzione è sospendere uno o più processi: le loro pagine vengono scritte su disco e i loro frame ridistribuiti agli altri, abbassando D fino a $D \leq m$. I processi sospesi vengono riattivati quando D scende sufficientemente.

Se invece $D \ll m$, esistono frame liberi inutilizzati e si possono ammettere nuovi processi senza rischio.

5.6 Implementazione approssimata del working set

Il calcolo esatto del working set richiederebbe di registrare ogni singolo riferimento alla memoria, il che è proibitivo in termini di overhead. Il sistema operativo usa quindi un'approssimazione basata sul bit di riferimento e sugli interrupt del timer, analoga a quanto visto per LRU approssimato.

Il meccanismo funziona come segue. Si sceglie un intervallo di campionamento τ (es. 5 000 referenze, corrispondenti a circa 1–5 ms su hardware moderno) e si mantiene per ogni pagina un registro di storia a k bit.

1. Ogni τ referenze scatta un interrupt del timer.
2. Il sistema operativo copia il bit di riferimento di ogni pagina nel bit più significativo del suo registro di storia, fa scorrere il registro di 1 bit verso destra e azzera il bit di riferimento.
3. Il registro a k bit rappresenta la storia degli ultimi k intervalli: bit a `1` in posizione j significa che la pagina è stata acceduta nell'intervallo j fa.
4. Una pagina appartiene al working set approssimato se almeno un bit del suo registro è a `1` (è stata acceduta in almeno uno degli ultimi k intervalli). Una pagina con registro tutto a `0` non fa parte del working set ed è candidata alla sostituzione.

Nota

L'approssimazione non è esatta per due ragioni:
- Non si conosce il momento preciso dell'accesso all'interno di un intervallo (solo se è avvenuto o no).
- La finestra effettiva è $k \times \tau$ referenze, non esattamente Δ.

L'accuratezza aumenta riducendo τ (interrupt più frequenti) e aumentando k (storia più lunga), al costo di un maggiore overhead per il sistema operativo.

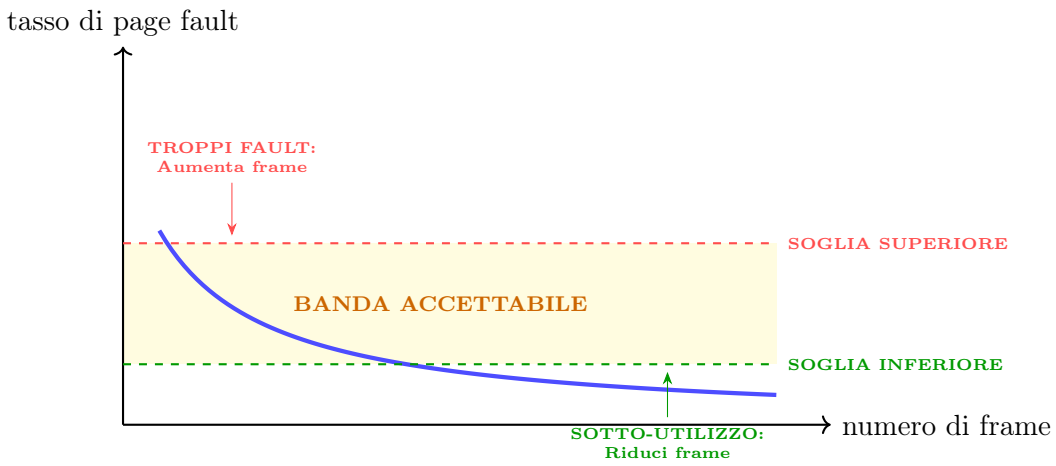
5.7 Controllo della frequenza di page fault (PFF)

Un approccio alternativo e complementare al working set per prevenire il thrashing è il controllo diretto della **frequenza di page fault** (*Page-Fault Frequency*, PFF).

Definizione

Il **controllo PFF** monitora il tasso di page fault di ciascun processo e aggiusta dinamicamente il numero di frame assegnati in base a soglie predefinite:
- Se il tasso di page fault del processo supera una **soglia superiore**, il processo ha troppo pochi frame: il sistema operativo gli assegna un frame aggiuntivo.
- Se il tasso scende al di sotto di una **soglia inferiore**, il processo ha frame in eccesso: il sistema operativo gliene sottrae uno e lo restituisce al pool dei liberi.

Graficamente, il tasso di page fault decresce all'aumentare del numero di frame assegnati (curva decrescente). Le due soglie definiscono una *banda di accettabilità*:



Osservazione

Il controllo PFF è più semplice del modello del working set perché non richiede di tracciare la finestra degli accessi: basta contare i page fault nell'unità di tempo. Tuttavia è *reattivo* piuttosto che *predittivo*: si accorge del thrashing solo dopo che il tasso ha già superato la soglia, mentre il working set può anticipare il problema stimando la domanda di frame prima che il thrashing si manifesti.
Nei sistemi reali, PFF e working set vengono spesso usati in combinazione: il working set per l'allocazione iniziale dei frame, PFF per gli aggiustamenti dinamici durante l'esecuzione.

5.8 Compressione della memoria

Una tecnica moderna alternativa allo swapping su disco per liberare frame è la **compressione della memoria** (*memory compression*).

Definizione

La **compressione della memoria** è una tecnica con cui il sistema operativo, invece di scrivere pagine su disco quando la memoria è scarsa, **comprime** più pagine in un singolo frame fisico, liberando i frame originali per altri usi. Le pagine compresse restano in RAM in forma compressa; se vengono accedute, vengono decomporesse on demand (generando un page fault speciale che non richiede I/O su disco).

Il meccanismo funziona come segue. Quando il numero di frame liberi scende al di sotto della soglia inferiore e il sistema deve liberare memoria:

1. Il sistema operativo seleziona alcune pagine vittime dalla lista delle pagine modificate (es. i frame 15, 3, 35 nell'esempio delle slide).
2. Le pagine selezionate vengono compresse con un algoritmo rapido (es. LZ4, zstd) e il risultato viene scritto in un singolo frame libero (es. frame 7), detto **frame compresso**.
3. I frame originali (15, 3, 35) vengono liberati e restituiti alla lista dei frame liberi.

4. Le PTE delle pagine compresse vengono marcate come non valide, con un riferimento alla loro posizione nel frame compresso.
5. Se una delle pagine compresse viene acceduta, si verifica un page fault; il sistema operativo decomprime quella pagina dal frame compresso e la carica in un frame libero.

Esempio

Si supponga che i frame 15, 3 e 35 contengano pagine modificate che non sono state accedute di recente. Il sistema li comprime nel frame 7:

- **Prima:** lista frame liberi $= [7, 2, 9, 21, 27, 16]$; lista frame modificati $= [15, 3, 35, 26]$.
- **Dopo:** lista frame liberi $= [2, 9, 21, 27, 16, 15, 3, 35]$; lista frame modificati $= [26]$; lista frame compressi $= [7]$.

La compressione ha liberato 3 frame (15, 3, 35) al costo di occuparne 1 (il frame 7 con il contenuto compresso), con un guadagno netto di 2 frame.

Osservazione

La compressione della memoria è particolarmente vantaggiosa sui dispositivi senza disco rotante o con storage lento (tablet, smartphone, laptop con eMMC). Su questi dispositivi lo swapping su disco è proibitivo in termini di latenza e usura dello storage; la compressione in RAM offre un compromesso efficace. macOS usa questa tecnica attivamente (*compressed memory*); Linux la implementa tramite il modulo `zswap`, che comprime le pagine in un'area di RAM dedicata prima di scriverle su swap.

6 Allocazione della Memoria del Kernel

6.1 Perché il kernel gestisce la memoria diversamente

Fino a questo punto abbiamo discusso la gestione della memoria per i processi utente: pagine di dimensione fissa, tabelle delle pagine, paginazione su richiesta. Il kernel del sistema operativo, tuttavia, gestisce la propria memoria in modo radicalmente diverso, per ragioni sia funzionali che di efficienza.

I processi utente ricevono frame dalla lista dei frame liberi, anche non contigui, e la MMU si occupa di renderli apparentemente contigui nello spazio virtuale. Questa flessibilità ha però un costo: se un processo richiede anche solo 1 byte di memoria dinamica, riceve un intero frame (tipicamente 4 KB), con uno spreco enorme per strutture dati piccole. Questo fenomeno si chiama **frammentazione interna**.

Il kernel deve allocare continuamente strutture dati di dimensioni molto variabili: descrittori di processo (*task_struct* in Linux, tipicamente 1–2 KB), socket, inode, semafori, buffer di I/O, tabelle delle pagine stesse. Molte di queste strutture sono più piccole di una pagina; allocare un intero frame per ciascuna sarebbe tremendamente inefficiente.

Esistono inoltre due requisiti aggiuntivi specifici del kernel che rendono impossibile usare la stessa strategia dei processi utente:

- **Memoria non paginabile.** La memoria del kernel è tipicamente *non* soggetta a paginazione su richiesta: non può essere mandata su disco. Se il gestore del page fault stesso o lo scheduler fossero su disco, il sistema si bloccherebbe irreparabilmente. La frammentazione nella memoria del kernel ha quindi conseguenze dirette e permanenti: memoria sprecata non può essere recuperata tramite swap.
- **Memoria fisicamente contigua per l'hardware.** Alcuni dispositivi hardware (schede di rete, controller disco con DMA, GPU) devono trasferire dati direttamente alla RAM senza passare per la MMU. Questi dispositivi richiedono che i buffer di memoria siano **fisicamente contigui**: non basta che siano contigui nello spazio virtuale del kernel, devono occupare frame fisici adiacenti. La paginazione standard non garantisce questa proprietà.

Osservazione

La distinzione tra contiguità virtuale e contiguità fisica è fondamentale. La MMU può rendere qualsiasi insieme di frame sparsi apparentemente contiguo nello spazio virtuale del kernel. Ma un dispositivo DMA che accede direttamente al bus fisico degli indirizzi non conosce la MMU: leggerà o scriverà agli indirizzi fisici specificati, che devono quindi essere davvero contigui in RAM.

Per questi motivi il kernel usa due allocatori specializzati: il **Buddy System** per allocazioni di grandi dimensioni (ordine della pagina) e lo **Slab Allocator** per allocazioni piccole e frequenti.

6.2 Il Buddy System

Il Buddy System è un algoritmo di allocazione della memoria che opera su un'area contigua di memoria fisica, suddividendola ricorsivamente in blocchi di dimensione potenza di due.

Definizione

Nel **Buddy System** l'intera area di memoria gestita viene considerata un unico blocco di dimensione 2^k byte (o pagine). Per soddisfare una richiesta di n byte:

1. Si calcola la più piccola potenza di due $2^j \geq n$.
2. Si cerca un blocco libero di dimensione 2^j.
3. Se non esiste un blocco di dimensione 2^j ma esiste uno di dimensione 2^{j+1}, questo viene **diviso in due metà uguali** (*buddies*) di dimensione 2^j. Una metà viene allocata, l'altra rimane libera.
4. Il processo di divisione viene ripetuto ricorsivamente finché non si ottiene un blocco della dimensione richiesta.

Alla deallocazione, se il blocco liberato e il suo *buddy* (l'altra metà dallo stesso split) sono entrambi liberi, vengono **fusi** (*coalesced*) in un unico blocco di dimensione doppia. La fusione avviene ricorsivamente verso l'alto fino a che non si trovano due buddy entrambi liberi.

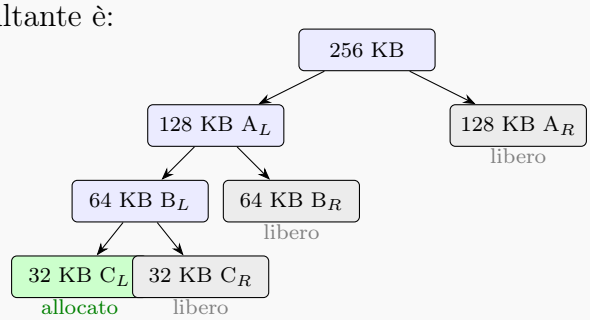
La struttura del Buddy System può essere rappresentata come un albero binario. Ogni nodo interno rappresenta un blocco che è stato diviso; i suoi due figli sono i buddy risultanti. Un blocco può essere fuso con il suo buddy solo se entrambi sono liberi e non ulteriormente divisi.

Esempio

Si consideri un'area di 256 KB. Arriva una richiesta di 21 KB.

1. La più piccola potenza di due $\geq$ 21 KB è 32 KB (2^5 KB).
2. Non esiste un blocco libero da 32 KB: si divide il blocco da 256 KB in due buddy da 128 KB (A_L e A_R).
3. Si divide A_L in due buddy da 64 KB (B_L e B_R).
4. Si divide B_L in due buddy da 32 KB (C_L e C_R).
5. Si alloca C_L (32 KB) per soddisfare la richiesta. Rimangono liberi: C_R (32 KB), B_R (64 KB), A_R (128 KB).

La struttura risultante è:



Ora arriva una deallocazione di C_L. C_L e il suo buddy C_R sono entrambi liberi $\rightarrow$ fusione in B_L (64 KB). B_L e il suo buddy B_R sono entrambi liberi $\rightarrow$ fusione in A_L (128 KB). A_L e il suo buddy A_R sono entrambi liberi $\rightarrow$ fusione nel blocco originale da 256 KB. La memoria è completamente recuperata.

Efficienza della fusione. Il Buddy System può determinare rapidamente se il buddy di un blocco è libero grazie a una proprietà aritmetica: se un blocco di dimensione 2^j inizia all'indirizzo a, il suo buddy inizia all'indirizzo $a \oplus 2^j$ (XOR bit a bit). Questo permette di trovare il buddy in tempo costante, senza scansioni.

Svantaggio: frammentazione interna. Il Buddy System soffre di **frammentazione interna**: una richiesta di 65 KB viene soddisfatta con un blocco da 128 KB, sprecando 63 KB. In generale, fino al 50% del blocco allocato può essere sprecato.

Osservazione

Il Buddy System è usato nel kernel Linux come allocatore di pagine fisiche (*page allocator*): gestisce l'allocazione di gruppi di pagine fisicamente contigue (da 2^0 a 2^{10} pagine) richieste da driver, slab allocator e operazioni di I/O. Il file `/proc/buddyinfo` su Linux mostra in tempo reale il numero di blocchi liberi per ogni ordine di potenza in ciascuna zona di memoria.

6.3 Lo Slab Allocator

Il Buddy System è efficiente per allocazioni dell'ordine delle pagine, ma è inadatto per le piccole strutture dati del kernel (di dimensione tipicamente 64–512 byte), che vengono create e distrutte migliaia di volte al secondo. Per queste lo Slab Allocator offre una soluzione ottimale.

Definizione

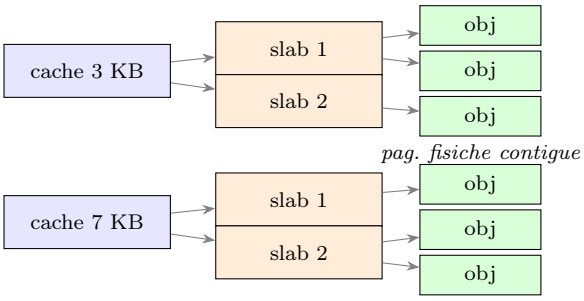
Lo **Slab Allocator** organizza la memoria del kernel in tre livelli gerarchici:

- **Oggetto (*object*):** un'istanza di una specifica struttura dati del kernel (es. un descrittore di processo, un inode, un semaforo).
- **Slab:** uno o più frame fisicamente contigui che contengono un array di oggetti dello stesso tipo, tutti della stessa dimensione. Ogni slab è ottenuto dal Buddy System.
- **Cache:** un insieme di slab dedicato a un unico tipo di struttura dati. Il kernel mantiene una cache separata per ogni tipo di oggetto frequentemente allocato (es. `kmem_cache` per `task_struct`, un'altra per `inode`, un'altra per `socket`, ecc.).

Ciclo di vita di una cache. Quando il kernel crea una nuova cache per un tipo di oggetto:

1. Viene allocato uno o più slab dal Buddy System.
2. Lo slab viene suddiviso in slot della dimensione esatta dell'oggetto. Tutti gli slot vengono **pre-inizializzati**: l'oggetto viene costruito (chiamando un costruttore specifico, se presente) e lasciato nello stato "libero".
3. Quando il kernel richiede un nuovo oggetto di quel tipo, lo Slab Allocator preleva il primo slot libero da uno slab parzialmente usato (o da uno slab vuoto se non esistono slot liberi in quelli parziali).
4. Quando il kernel libera l'oggetto, lo slot torna disponibile nella cache, pronto per essere riutilizzato senza dover essere re-inizializzato da zero.

Esempio numerico. Si supponga che la struttura `task_struct` (descrittore di processo) occupi 1.7 KB e che il kernel usi pagine da 4 KB. Uno slab di tre pagine contigue (12 KB) può ospitare $\lfloor 12/1.7 \rfloor = 7$ oggetti, con $12 - 7 \times 1.7 = 0.1$ KB di spazio interno non usato — frammentazione trascurabile.



6.4 Vantaggi dello Slab Allocator

Lo Slab Allocator offre due vantaggi fondamentali rispetto agli allocatori generici.

Assenza di frammentazione interna. Ogni cache è dedicata a un unico tipo di oggetto, e ogni slab è suddiviso in slot della dimensione *esatta* di quell'oggetto. Non vi è spazio sprecato all'interno degli slot (a differenza del Buddy System, che arrotonda alla potenza di due più vicina). La piccola quantità di spazio non usata alla fine di ogni slab (dovuta alla divisione intera) è trascurabile.

Allocazione e deallocazione estremamente rapide.

- Gli oggetti sono pre-costruiti e pre-inizializzati nella cache. Allocare un oggetto significa semplicemente estrarre uno slot libero dalla cache: un'operazione $O(1)$ senza necessità di inizializzazione.
- Deallocare un oggetto significa reinserire lo slot nella cache, rendendolo immediatamente disponibile per la prossima allocazione: ancora $O(1)$.
- Nelle strutture dati del kernel che vengono create e distrutte migliaia di volte al secondo (es. descrittori di processo durante fork/exec, socket durante connessioni di rete), questo risparmio si accumula in un guadagno di prestazioni significativo.

Procedura di allocazione. Quando arriva una richiesta di un oggetto di tipo T:

1. Lo Slab Allocator cerca uno slot libero in uno slab **parzialmente usato** della cache di T.
2. Se non esistono slab parzialmente usati, cerca uno slot in uno slab **completamente vuoto**.
3. Se non esistono slab vuoti, alloca un nuovo slab dal Buddy System, lo suddivide in slot, pre-inizializza gli oggetti e restituisce il primo slot.

Nota

Lo Slab Allocator è stato introdotto in Solaris nel 1994 da Jeff Bonwick e adottato da Linux nel 1999. Linux offre oggi tre varianti: `SLAB` (l'originale), `SLUB` (la variante moderna, predefinita dalla versione 2.6.23, più semplice e performante) e `SLOB` (una versione minimale per sistemi embedded con poca RAM). Le strutture allocate dallo slab allocator sono visibili su Linux tramite il file `/proc/slabinfo`.

Osservazione

La combinazione Buddy System + Slab Allocator costituisce la spina dorsale della gestione della memoria del kernel Linux. Il Buddy System garantisce la disponibilità di blocchi di pagine fisicamente contigui per le esigenze hardware e per il Slab Allocator stesso; il Slab Allocator usa quei blocchi per costruire pool di oggetti piccoli, eliminando la frammentazione e massimizzando la velocità di allocazione. I due sistemi sono complementari e si occupano di scale dimensionali diverse: pagine intere (Buddy) vs oggetti sub-pagina (Slab).